LLM RAG Agent lesson plan

Ver 3.0

Table of Contents

1. Overview

This textbook summarizes the main learning content by topic. In addition to the textbook, actual lectures utilize the author's blog, GitHub, huggingface, and papers.

ÿ mac999/AI_foundation_tutorial

ÿ mac999/LLM-RAG-Agent-Tutorial: LLM-RAG-Agent-Tutorial

2. Natural Language Processing (NLP)

2.1 Introduction

Natural Language Processing (NLP) is a field of study that enables computers to understand and interpret human language. NLP deals with the technology that structures and analyzes unstructured data, such as text or speech, to extract meaning and perform various tasks based on this data. Representative applications include machine translation, question-and-answer systems, document summarization, sentiment analysis, and document classification.

2.2 Basic NLP Concepts

A word is the smallest unit of text that carries meaning. It is the basic unit people use to communicate their thoughts through language, and in natural language processing, words form sentences when analyzing text. It is common to divide the work into units.

In natural language processing, a token is the result of dividing the input text into units. A token is not necessarily a word unit. It's not necessary, and in some cases, it can be broken down into subwords or even characters. For example, it's possible to break down the word "unbelievable" into "un," "believe," and "able," semantically. This is accomplished using a tool called a tokenizer, which helps the model generalize better.

Embedding refers to the process of mapping discrete text information, such as words, sentences, or documents, to a continuous vector space. Natural language is inherently symbolic and discrete, so it must be converted into a fixed-dimensional continuous numerical vector to be effectively input into a machine learning model. The vector created at this time is It's called embedding.

2.3 Embedding Model and Learning Method

Representative embedding models include Word2Vec, GloVe, and FastText.

Among these, Word2Vec is an embedding method proposed by Google in 2013, and is similar to Skip-gram.
CBOW (Continuous Bag of Words) has two learning structures. Skip-gram takes a central word as input and predicts surrounding words. For example, in the sentence "The quick brown fox jumps," if the central word is "fox" and the window size is set to 2, the surrounding words would be "brown" and "jumps." The model learns to identify "brown" and "jumps" by looking at "fox."

The equation is as follows:

$$\text{maximize} \prod_{-c \le j \le c, j \neq 0} P(w_{t+j} | w_t)$$

Here, ccc is the window size, Wt is the center word, and Wt+j is the surrounding word.

Embedding models are mainly trained using unsupervised learning methods on large-scale text data.
Word2Vec minimizes the loss for predicting surrounding words.

A representative example of embedding learning code is as follows.

```
# Word2Vec Skip-gram simple pseudocode for
center_word, context_words in dataset:
    center_vec = word_embedding(center_word) for context in
    context_words:
        context_vec = word_embedding(context) loss +=
        negative_sampling_loss(center_vec, context_vec)
    loss.backward()
    optimizer.step()
```

2.4 Key NLP Technologies

Named Entity Recognition (NER) is a technology that identifies specific entities in a sentence and categorizes them into categories such as person, organization, or location. For example, in the sentence "Apple Inc. was founded by Steve Jobs," "Apple Inc." is classified as the organization, and "Steve Jobs" is classified as the person.

Text classification is the task of predicting which category a given sentence belongs to. Representative examples include sentiment analysis and news article classification.

Machine translation is a technology that converts sentences written in one language into another. In the past, statistical machine translation was the mainstream, but now
Transformer-based neural machine translation is the trend.

Text Summarization extracts only the key points from a long document or rephrases sentences.
It is a technique of generating and summarizing. Extractive summarization and generative summarization
It is divided into abstract summarization.

Question Answering is a system that provides accurate answers to entered questions.
It refers to a system that ranges from simple information retrieval-based systems to complex language understanding-based generative responses.
There are many different forms of systems.

2.5 N-gram

2.5.1 concept

N-gram is a concept that means a group of N consecutive **words or character units .** Natural language
In processing (NLP), it is mainly used to model **context** or **construct language models .**
For example, 2-gram analyzes words in groups of two, and 3-gram analyzes words in groups of three.

2.5.2 type

Unigram (1-gram): word unit
Example: ["I", "eat", "eat"]

Bigram (2-gram): Two consecutive words
Example: [("I", "rice"), ("rice", "ate")]

Trigram (3-gram): three consecutive words
Example: [("I", "eat", "eat")]

2.5.3 Usage examples

Language modeling: Used to predict the next word or calculate the probability of a sentence. For example, the Bigram model learns the probability of "ÿÿ" appearing after "ÿÿ."

Spam filtering: If a specific n-gram combination appears frequently in spam, it can be filtered based on that probability.

Auto-complete/typo correction: Predicts the next word based on the previous n-1 words.

Calculating text similarity: You can decompose sentences into n-grams and calculate similarity based on the overlap ratio.

2.6 Accuracy indicators and calculation methods

Several metrics are used to quantitatively evaluate model performance. Each metric is defined by the following formula:

Accuracy is the proportion of the total data that is predicted correctly.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Here,

TP(True Positive): Predicting the truth as true

TN(True Negative): Predicting falsehood as false

FP(False Positive): Predicting false as true

FN(False Negative): Predicting true as false

For example, if you got 90 out of 100 correct, your Accuracy is 0.9.

Precision is the percentage of what the model predicted to be true that was actually true.

$$\text{Precision} = \frac{TP}{TP + FP}$$

For example, if you predicted 50 to be true and 45 of them were actually true, the Precision is 0.9.

Recall is the proportion of actual true data that the model correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score is the harmonic mean of Precision and Recall.

$$\mathrm{F1} = 2 \times \frac{\mathrm{Precision} \times \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

A simple example is as follows:

When the total data is 100,

If TP = 40, TN = 50, FP = 5, FN = 5,

Accuracy is (40+50)/100=0.9(40+50)/100 = 0.9, Precision is 40/

(40+5)=0.88840/(40+5) = 0.888, Recall is 40/(40+5)=0.88840/

(40+5) = 0.888, and F1 Score is about 0.888.

If we calculate it with code, it is as follows.

```
# Simple Precision, Recall, and F1 calculation example

precision = TP / (TP + FP)

recall = TP / (TP + FN)

f1 = 2 * (precision * recall) / (precision + recall)
```

2.7 BLEU and ROUGE indices

BLEU (Bilingual Evaluation Understudy) is a metric for evaluating machine translation results. It is based on the degree of n-gram agreement between the translation result and a reference translation.

The BLEU score is calculated using the following formula:

$$\mathrm{BLEU} = \mathrm{BP} \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Here, BP is the Brevity Penalty given when the translation is shorter than the reference translation.

For example, if the reference sentence is "the cat is on the mat" and the machine translation is "the cat is on mat", the 1-gram precision is 0.8, since 4 out of 5 matches.

ROUGE (Recall-Oriented Understudy for Gitting Evaluation) is a metric for evaluating summarization performance. It is based on the degree of n-gram overlap between the baseline and model summaries, and primarily focuses on recall.

ROUGE-1 is calculated by the following formula:

$$\text{ROUGE-1} = \frac{\text{overlapping unigrams}}{\text{total unigrams in reference}}$$

If the baseline summary is "the cat sat on the mat" and the model summary is "the cat sat mat", there are 4 matching words: "the", "cat", "sat", and "mat", and the baseline is 6 words, so ROUGE-1 is 4/6ÿ0.6664/6 = 0.666.

The example code for calculating BLEU and ROUGE indices is as follows.

```
# Simple BLEU score calculation example

import nltk

reference = [['the', 'cat', 'is', 'on', 'the', 'mat']]

candidate = ['the', 'cat', 'is', 'on', 'mat']

score = nltk.translate.bleu_score.sentence_bleu(reference, candidate)
```

2.8 BLUE function pseudocode

2.8.1 outline

Input: candidate sentence, reference sentence

1. Tokenize candidate and reference.

2. For each 1-gram, 2-gram, 3-gram, and 4-gram: - Create n-grams of the candidate. - Create n-grams of the reference. - Count the number of matches between the candidate n-gram and the reference n-gram. - precision = number of matches / total number of n-grams of the candidate

3. Take the log of all n-gram precisions and average them.

4. Geometric mean = exp(log mean)

5. Calculating Brevity Penalty (BP):

- If candidate length < reference length: BP = exp(1 - reference_len / candidate_len)
- If candidate length >= reference length: BP = 1

6. BLEU = BP × Geometric Mean

Output: BLEU score

2.8.2 Step-by-step BLEU calculation process using examples

Example sentences

- Reference: “the cat is on the mat”
- Candidate: “the cat is on the mat”

Tokenization:

- reference = ["the", "cat", "is", "on", "the", "mat"]
- candidate = ["the", "cat", "is", "on", "mat"]

2.8.3 1-gram precision calculate

1-grams of candidate:

“the”, “cat”, “is”, “on”, “mat”

1-grams of reference:

"the", "cat", "is", "on", "the", "mat"

Matches :

"the", "cat", "is", "on", "mat" ÿ

Matches all 5 (Note: "the" is in the reference twice, but only in the candidate once, so it passes)

precision (1-gram) = 5 / 5 = 1.0

Calculation 2.8.4 2-gram precision

2-grams of candidate:

“the cat”, “cat is”, “is on”, “on mat”

2-grams of reference:

“the cat”, “cat is”, “is on”, “on the”, “the mat”

Matches :

“the cat”, “cat is”, “is on” (3 in total)

precision (2-gram) = 3 / 4 = 0.75

Calculation 2.8.5 3-gram precision

3-grams of candidate:

“the cat is”, “cat is on”, “is on mat”

3-grams of reference:

“the cat is”, “cat is on”, “is on the”, “on the mat”

Matches :

“the cat is”, “cat is on” (2 total)

precision (3-gram) = 2 / 3 ÿ 0.6667

Calculation 2.8.6 4-gram precision

4-grams of candidate:

“the cat is on”, “the cat is on mat”

4-grams of reference:

“the cat is on”, “cat is on the”, “is on the mat”

Matches :

"the cat is on" (1 total)

precision (4-gram) = 1 / 2 = 0.5

2.8.7 log exp average and calculation

Find the log of each precision:

log(1.0) = 0

log(0.75) ÿ -0.28768

log(0.6667) ÿ -0.40547

log(0.5) ÿ -0.69315

Their average:

$$\frac{0 - 0.28768 - 0.40547 - 0.69315}{4} \approx -0.34658$$

exp(average):

exp(ÿ0.34658)ÿ0.707

2.8.8 Brevity Penalty (BP) calculate

candidate length = 5

reference length = 6

Since the candidate is shorter, we calculate the BP:

$$BP = \exp(1 - 6/5) = \exp(1 - 1.2) = \exp(-0.2) \approx 0.8187$$

2.8.9 BLEU Final score calculation

BLEU=BP×Geometric Mean=0.8187×0.707ÿ0.579

That is, the BLEU score for this example is approximately **0.579 .**

2.8.10 Simple code example(Python + Numpy)

```
import numpy as np
from collections import Counter

def bleu_score(candidate, reference):
    candidate = candidate.split()
    reference = reference.split()

    precisions = []
    for n in range(1, 5):
        cand_ngrams = Counter([tuple(candidate[i:i+n]) for i in range(len(candidate)-n+1)])
        ref_ngrams = Counter([tuple(reference[i:i+n]) for i in range(len(reference)-n+1)])

        overlap = sum(min(count, ref_ngrams[ngram]) for ngram, count in
cand_ngrams.items())
        total = sum(cand_ngrams.values())
        precisions.append(overlap / total if total > 0 else 0)

    if min(precisions) > 0:
        log_precisions = np.mean([np.log(p) for p in precisions])
        geo_mean = np.exp(log_precisions)
    else:
        geo_mean = 0

    c, r = len(candidate), len(reference)
    bp = np.exp(1 - r/c) if c < r else 1.0

    return bp * geo_mean
```

```
# Example
candidate = "the cat is on mat"
reference = "the cat is on the mat"

print(f"BLEU score: {bleu_score(candidate, reference):.4f}") output:

BLEU score: 0.5790
```

2.9 Conclusion

Natural language processing (NLP) is a technological system that quantifies words and performs various tasks based on their numerical representations. Embedding models convert text into continuous vectors, various neural network models infer context and meaning, and quantitatively evaluate performance through accuracy metrics. In NLP, embedding techniques such as Word2Vec, GloVe, and FastText, as well as large-scale pre-trained language models like BERT and GPT, enable context-based processing. Evaluation metrics such as BLEU, ROUGE, and F1 Score serve to objectively measure model quality.

3. Regular expressions

3.1 Overview

Regular expressions are used to find, replace, or separate specific patterns in a string.

It is a string search tool for complex text. Regular expressions are used in various programming languages.

It simplifies processing tasks, primarily data validation, log analysis, and text parsing.

It is widely used in the back.

3.2 History

The concept of regular expressions was developed by American mathematician Stephen First by Stephen Kleene was introduced. He laid the theoretical foundation for regular languages through mathematical expressions related to the theory of finite automata. Later, regular expressions were adopted by text processing tools (e.g., grep, sed, etc.) of the Unix operating system, evolving into a practical tool. Today, regular expressions provide standard functionality in various languages, including Python, JavaScript, Java, Perl, and Ruby.

3.3 Main regular expression syntax and examples

Regular expressions construct patterns using various symbols and metacharacters. Below is the main grammar and simple

This is an example.

grammar	explanation	example	explanation
. any single character ac			Matches 'abc', 'axc'
^ Start of string ^a			Matches 'apple', X for 'banana'
**$** end of string e$ matches 'apple', 'title'			
[] Matches the character set [aeiou] vowel			
[^] Excluded character set [^0-9] Non-numeric characters			
*	0 or more repetitions of a*		Matches all '', 'a', and 'aa'
+	1 or more repetitions a+		'a', 'aa' match, '' is X
?	0 or 1	a?	'', 'a' matches
{n} Repeat exactly n times a{3} **{n,}**			Matches only 'aaa'
Repeat at least n times a{2,} **{n,m}**			Matches 'aa', 'aaa', ...
Repeat n~m times a{1,3}			Matches 'a', 'aa', 'aaa'
\d numeric character		\d+ matches '123', '5', etc.	
\w word character **\s**		\w+ matches 'abc123', '_', etc.	
space character		\s+	Matches ' ', '\t', '\n', etc.
() grouping		(abc)+ matches 'abc', 'abcabc'	

3.4 Examples of what you can do with regular expressions

Extract phone numbers: \d{2,3}-\d{3,4}-\d{4}

Check if a specific word is included: \bpython\b

Remove HTML tags: <[^>]+>

3.5 How to use Python's re library

In Python, you can use the re module to handle regular expressions. The main functions are as follows:

```
import re

# search
re.search(r"apple", "I like apple") # Returns a matching object

#
Matching re.match(r"apple", "apple pie") # Attempt to match from the beginning of the string

# Find all
re.findall(r"\d+", "There are 24 cats and 3 dogs.") # Returns ['24', '3']

# Substitution
re.sub(r"cat", "dog", "I have a cat") # 'I have a dog'

# Split
re.split(r"\s+", "split by space or tab") # ['split', 'by', 'space', 'or', 'tab']
```

3.6 Conclusion

Regular expressions are a powerful tool that allows you to perform complex string processing tasks concisely. A thorough understanding of regular expression syntax and practical practice will enable efficient text processing in a variety of fields, including data analysis, web crawling, and log processing.

4. Transformer structure and operating mechanism

4.1 Introduction

Transformers are essential structures used in the fields of natural language processing and generative AI, and encoders in particular
It plays a key role in understanding the input sequence and capturing contextual information. In this lesson,
Focusing on the transformer's operating principles, we analyze its core components and PyTorch-based implementation code.
Explain.

4.2 Transformer Attention Operation Mechanism

This chapter describes the self-attention and cross-attention mechanisms, which are the most important in transformers.
Cross-attention is mainly used in the decoder part of the sequence-to-sequence (Seq2Seq) model, and it is used to learn the
original sequence learned by the encoder.
It plays a role in effectively combining context information and information from the target sequence that the decoder is currently generating.
In particular, understanding the complex relationships between different languages, such as English-Korean translation, and
It is an essential mechanism for improving quality.

This chapter describes step by step how a transformer works when translating from English to Korean.
It is.

4.2.1 Description of key variables in the training dataset

The Transformer training dataset for English-Korean translation is used to help the model learn mappings between languages.
It contains various necessary components. Key variables and examples are as follows.

4.2.2 Source Sequence - English sentences

These are the original language (English) sentences that the model must learn. These sentences go through a tokenization process.
It is converted into a sequence of words or subwords.

ÿ **Example variable name:** english_sentences or source_texts

ÿ **Data type:** List of Strings.

ÿ **Example data:**

```
english_sentences = [
    "I love machine learning.",
```

"How are you doing today?",

“The cat is sleeping on the sofa.”,

“Artificial intelligence is transforming the world.”,

“Please translate this sentence.”

]

4.2.3 Target Sequence - Korean sentence

These are the target language (Korean) sentences that the model must translate. These sentences also undergo tokenization and are used to compare model outputs and calculate loss during training.

ÿ **Variable name example:** korean_sentences or target_texts ÿ **Data type:** List of Strings.

ÿ **Example data:**

korean_sentences = [

"I love machine learning."

"How are you today?",

"The cat is sleeping on the sofa.",

"Artificial intelligence is changing the world."

"Please translate this sentence."

]

4.2.4 Tokenized Source Sequence

A sequence of integer IDs obtained by tokenizing original English sentences. Each integer maps to a specific word or subword in the vocabulary.

ÿ **Example variable name:** tokenized_source_ids or encoder_input_ids ÿ **Data type:** List of Lists of Integers or

It is a tensor.

ÿ **Example data (fictitious token ID):**

```
# Example of mapping a virtual Korean vocabulary
# {"<pad>": 0, "<bos>": 1, "<eos>": 2, "I": 100, "Machine Learning": 101, "I love": 102, ...}

# Decoder input (Shifted Right)
# During training, the decoder uses the previous output (shifted right) as input to predict the next token.
decoder_input_ids = [
    [1, 100, 101, 102, 2], # "<bos> I love machine learning." -> [<bos>, I love machine learning,
I love you, <eos>]
    [1, 103, 104, 105, 106, 2], # "<bos> How are you today?" -> [<bos>, today, how,
How are you, ?, <eos>]
    #  ...
]

# Decoder Label (Expected Output)
# The actual next token that the model should predict.
decoder_label_ids = [
    [100, 101, 102, 2, 0], # "I love machine learning.<eos><pad>" (Padding taken into account)
    [103, 104, 105, 106, 2, 0], # "How are you today?<eos><pad>"
    #  ...
]
```

ÿ <bos> (Beginning-of-Sentence): A special token indicating the beginning of a sentence. It is used in decoder input.

Used as the first token to start generating translations.

ÿ <eos> (End-of-Sentence): A special token that indicates the end of a sentence. The decoder uses this token.

Once created, the translation is considered complete.

ÿ Shifted Target Sequence: The decoder of the transformer

It operates autoregressively, meaning that at each time step, it computes the previously generated

It predicts the next token based on the tokens. Therefore, during training, the input of the decoder is the actual target.

A form shifted one space to the right from the sequence (including <bos> in tokenized_target_ids)

, and the target output (label) of the decoder is the actual target sequence (<bos> of tokenized_target_ids

(excluding <eos> and including <eos>).

4.2.5 Tokenized TargetSequence

This is an integer ID sequence obtained by tokenizing the target Korean sentences. This sequence is the input of the decoder and

Used as an output label.

ÿ **Example variable names:** tokenized_target_ids or decoder_input_ids (decoder input),

decoder_label_ids (decoder output labels)

ÿ **Data type:** List of Lists of Integers or

It is a tensor.

ÿ **Example data (fictitious token ID):**

```
# Example of mapping a virtual Korean vocabulary
# {"<pad>": 0, "<bos>": 1, "<eos>": 2, "I": 100, "Machine Learning": 101,
"I love you": 102, ...}


# Decoder input (Shifted Right)
# During training, the decoder uses the previous output (shifted right) as input to generate the next token.
Predict.
decoder_input_ids = [
    [1, 100, 101, 102, 2], # "<bos> I love machine learning." -> [<bos>,
I love machine learning <eos>
    [1, 103, 104, 105, 106, 2], # "<bos> How are you today?" -> [<bos>, today,
How are you doing?, <eos>]
    # ...
```

```
]

# Decoder Label (Expected Output)
# The actual next token that the model should predict.
decoder_label_ids = [
    [100, 101, 102, 2, 0], # "I love machine learning.<eos><pad>" (Padding taken into account)

    [103, 104, 105, 106, 2, 0], # "How are you today?<eos><pad>"
    #  ...
]
```

ÿ **<bos> (Beginning-of-Sentence):** A special token indicating the beginning of a sentence. Decoder
Used as the first token of the input to start generating the translation.

ÿ **<eos> (End-of-Sentence):** A special token indicating the end of a sentence. The decoder
Once a token is generated, the translation is considered complete.

ÿ **Shifted Target Sequence:** The decoder of the transformer
It operates autoregressively, meaning that at each time step, it computes the previously generated
It predicts the next token based on the tokens. Therefore, during training, the input of the decoder is the actual
The target sequence is shifted one space to the right (including <bos> in tokenized_target_ids), and
the target output (label) of the decoder is the actual target.
It becomes a sequence (including <eos> but excluding <bos> in tokenized_target_ids).

4.2.6 Padding Mask

Padding tokens (<pad>) added to adjust the sequence length are used to mask them from affecting attention calculations. Since these tokens are not part of the actual content, the model is prevented from attending to them.

ÿ **Example variable names:** source_padding_mask, target_padding_mask ÿ

Data type: Boolean tensor or binary tensor (0 or 1). ÿ **Example data (based on a hypothetical token ID):**

```
# Assuming padding to a maximum sequence length of 10
# `tokenized_source_ids` example:
# [3, 4, 5, 6, 2, 0, 0, 0, 0, 0] (original length 5)

source_padding_mask = [
    [1, 1, 1, 1, 1, 0, 0, 0, 0, 0], # 1: valid token, 0: padding token
    [1, 1, 1, 1, 1, 1, 0, 0, 0, 0], # "How are you doing today?" (length 6)
    #  ...
]

# `decoder_input_ids` example:
# [1, 100, 101, 102, 2, 0, 0, 0, 0, 0] (original length 5)

target_padding_mask = [
    [1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
    [1, 1, 1, 1, 1, 1, 0, 0, 0, 0],
    #  ...
]
```

ÿ **Role:** Before calculating Softmax in the attention mechanism, it corresponds to the padding position. Set the score (logits) to -inf (negative infinity) so that the weight at that position becomes 0.

4.2.7 Look-Ahead Mask - for decoder self-attention

It is used in the decoder's self-attention layer. Since the decoder operates autoregressively, future tokens must be masked to prevent them from "looking ahead" when predicting the next token at a given time step. This is to maintain the principle that translation generation must be sequential.

ÿ **Example variable name:** look_ahead_mask

ÿ **Data type:** Boolean tensor or binary tensor (in the form of an upper triangular matrix).

ÿ Example:

Assuming we have a sequence [A, B, C, D], the look-ahead mask is as follows:

It is created.

[[1, 0, 0, 0], # A can only see A.

[1, 1, 0, 0], # B can see A, B.

[1, 1, 1, 0], # C can see A, B, C.

[1, 1, 1, 1]] # D can see A, B, C, D.

ÿ **Role:** Prevents each position in the decoder from paying attention to the token after the current position.

4.3 Transformer Attention Computational Learning Process (English-Korean Translation)

Cross-attention occurs in the second attention layer of the Transformer decoder. This process is a key step, where the decoder retrieves contextual information (i.e., the meaning of the original sentence) from the encoder and combines it with the context of the currently predicted Korean translation.

home:

ÿ **Input (Source):** English sentence "I love machine learning."

ÿ **Output (Target):** Korean sentence "I love machine learning." ÿ These sentences have already been tokenized and embeddings to form vector representations.

Assuming it has been converted.

ÿ For convenience of explanation, the embedding dimension d_model is assumed to be 4 and the sequence length is assumed to be short.

4.3.1 1 Step 1: Prepare the encoder output and decoder input

1. **Final output of the encoder (Key & Value Source)**

- The English sentence "I love machine learning." uses multiple layers of transformer encoders.

  Pass through.

- The final layer output of the encoder is each input token (I, love, machine, learning,

  A vector sequence containing rich contextual information about the <eos> (e.g. [vec_I, vec_love,

  vec_machine, vec_learning, vec_eos]).

- This encoder output is **Key (K)** and **Value (V)** in the cross attention layer.

  It is used as a source. That is, K = Encoder_Output and V =

  Encoder_Output.

- **Example:** Encoder_Output is for each word in the English sentence "I love machine learning."

  These are vectors that reflect the corresponding context.

2. **Current input of the decoder (Query Source)**

- The decoder generates Korean translation sentences. During training, **up to the previous time step

  It takes as input the correct token (or <bos> token)**. This is called "Shifted Right".

  It's called input.

- For example, in the process of generating “I love machine learning,” the decoder has already

  " <bos> I have created machine learning" and the next word is "I love you"

  If we have to predict, the current input to the decoder is "<bos> I do machine learning".

- The input sequence of this decoder is the first self-attention layer inside the decoder.

  A new vector sequence (e.g. [dec_vec_bos,

  dec_vec_I, dec_vec_machine learning]) is converted to.

- The intermediate output of this decoder is **Query (Q)** in the cross attention layer.

  It is used as a source. That is, Q = Decoder_Self_Attention_Output.

- **Example:** Decoder_Self_Attention_Output is each of the Korean sentences "I do machine learning"

  These are vectors that reflect the context corresponding to the token.

4.3.2 Step 2: Linear Transformation Q, K, V (Linear Projections)

The cross-attention layer has three learnable weight matrices W_Q, W_K, and W_V. These are Projects a vector of embed_dim size into another representation space of the same size.

- **Creating a query (Q) is as follows:**
  - Q = Decoder_Self_Attention_Output @ W_Q.
  - **Example:** [dec_vec_bos, dec_vec_I, dec_vec_machine learning] are each multiplied by W_Q

    Form a new query vector [q_bos, q_me, q_machine learning]. These queries are

    "I need to focus on some English words to be able to predict the next Korean word well.

    It contains the question, "Is there anything like it?"
- **Key (K) generation is as follows:**
  - K = Encoder_Output @ W_K.
  - **Example:** [vec_I, vec_love, vec_machine, vec_learning, vec_eos] each with W_K

    Multiply them to form a new key vector [k_I, k_love, k_machine, k_learning, k_eos].

    These keys are like saying, "I have this information (English words)!"
- **Value (V) creation is as follows:**
  - V = Encoder_Output @ W_V.
  - **Example:** [vec_I, vec_love, vec_machine, vec_learning, vec_eos] each with W_V

    Multiply them to form a new value vector [v_I, v_love, v_machine, v_learning, v_eos].

    These values are the "real content" of the information provided by the keys.

4.3.3 Step 3: Calculate AttentionScore (Attention Scores)

How does each query vector (information needed to generate Korean) relate to all key vectors (information on the original English text)?

Measures whether something is "related". Computes similarity using the dot product.

- **Calculation:** Scores = Q @ KT (transpose product of query matrix and key matrix).

- **Example (conceptual):**
  - score_I_I = q_I . k_I (How closely related is "I" to the English "I"?).
  - score_I_love = q_I . k_love (How related is "I" to the English "love"?)

    Is it deep?)
  - ...
  - score_machine_learning_machine = q_machine_learning. k_machine ("machine learning" is English)

    How closely related is it to "machine"?)
  - score_machine_learning = q_machine_learning. k_learning ("machine learning" in English)

    How closely related is it to "learning"?)

The result is a score matrix of size (decoder_sequence_length x encoder_sequence_length).

4.3.4 4 Step: Scaling

Divide the inner product values (Scores) by the square root of the dimension (d_k) of the key vector. This is because the inner product values become too large.

Prevents vanishing gradients of the softmax function and improves learning

It is a way to increase stability.

- **Calculation:** Scaled_Scores = Scores / sqrt(d_k).

4.3.5 5 Step: Masking

If the original English sentence contains padding tokens (pad_token), those padding tokens have no actual meaning.

Since there is no effect on the attention calculation, it is masked.

- **Application:** Set the value of the column corresponding to the padding token in the Scaled_Scores matrix to -inf (negative infinity).

  Set it up.
- **Example:** If the encoder input is "I love machine learning. <pad> <pad>" and vec_pad is

  If present, the score column corresponding to k_pad is changed to -inf.
  - Scaled_Scores[:, index_of_pad_token] = -inf.

4.3.6 Step 6: Softmax and Attention Weights

Applying the softmax function to the scaled scores creates weights in the form of a probability distribution for each key.
We get. The sum of these weights becomes 1.

- **Calculation:** Attention_Weights = Softmax(Scaled_Scores, dim=-1).
- **Meaning:** How much each decoder token (query) should focus on each token (key) of the encoder.
  This is the probability that the Korean token "I" is generated when the English original text "I" is generated.
  This is the formula that gives the highest weight.
- **Example:**
  - Korean query q_I will have the highest attention weight on English key k_I.
  - Korean query q_machine learning has high attention to English keys k_machine, k_learning
    It will have weights.

4.3.7 7 Step: Weighted Sum and Context Vector

The calculated attention weights are multiplied by the value vector and then added together to generate the final output.

- **Calculation:** Context_Vector = Attention_Weights @ V.
- **Meaning:** For each decoder token (query), a weighted sum of all tokens (values) from the encoder is calculated.
  A context vector is created. This vector is used to generate the corresponding decoder token.
  It summarizes the relevant information on the encoder side. The model uses this context vector to
  Effectively reflects information from the original text into the Korean translation.
- **Example:**
  - context_for_I = (attn_weight_I_I * v_I) + (attn_weight_I_love *
    v_love) + ...
  - context_for_machine_learning = (attn_weight_machine_learning_I * v_I) +
    (attn_weight_machine_learning_love * v_love) + ...

This Context_Vector becomes a matrix of size (decoder_sequence_length x embed_dim).

4.3.8 8 Step: Final linear transformation

If you use Multi-Head Attention (most transformers do), each
The Context_Vectors generated from the head are concatenated and finally formed into a single linear layer.
It is projected back to the original embed_dim dimensions through W_O.

- **Calculation:** Final_Output = Concatenate(Head1_Context, Head2_Context, ...) @
  It's W_O.
- This Final_Output is the next sub-layer of the decoder (usually feedforward
  is transmitted to the network.

4.3.9 9 &Step: Loss Calculation and Backpropagation (Loss Calculation Backpropagation)

- The final output of the decoder is Korean, which passes through the linear layer (W_proj) and the softmax function.
  It is converted to a probability distribution (logits) of the vocabulary size (vocab_size).
- This probability distribution represents the distribution of the next Korean token predicted by the decoder.
- The model's predictions (predicted_probabilities) are the actual correct answers in Korean
  It is compared with tokens (decoder_label_ids).
- **Loss Function:** Cross-Entropy Loss is mainly used to compare predictions and actual results.
  Measure the difference between correct answers.
  - Loss = CrossEntropyLoss(predicted_probabilities,
    actual_target_token_ids).
- **Backpropagation:** The calculated loss is applied to all the weights of the neural network (embedding weights,
  It is used to update the Q/K/V/O matrix, feedforward network weights, etc.
  This is done via an optimizer (e.g. Adam).
  - Loss.backward() (gradient calculation).
  - optimizer.step() (weight update).

4.4 Cross-Attention Python Pseudocode Implementation

The following is a pseudocode example of a Transformer cross-attention layer implemented in Python (PyTorch). In actual Transformer models, it is implemented as part of Multi-Head Attention.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class CrossAttention(nn.Module):
    def __init__(self, embed_dim, num_heads):
        """
        Initialize the cross attention layer.
        Args:
            embed_dim (int): The dimension of the input
            embedding. num_heads (int): The number of attention heads.
        """
        super(CrossAttention, self).__init__()
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads # Dimensions of each head.
        assert (
            self.head_dim * num_heads == embed_dim ),
        "embed_dim must be divisible by num_heads."

        # Linear projection layer for Query, Key, and Value. #
        Query projection for the output of the decoder.
        self.query_proj = nn.Linear(embed_dim, embed_dim) # Key
        projection for the output of the encoder.
        self.key_proj = nn.Linear(embed_dim, embed_dim) #
        Value projection for the encoder's output.
        self.value_proj = nn.Linear(embed_dim, embed_dim) # The
        final linear projection layer to combine the outputs of all heads.
        self.out_proj = nn.Linear(embed_dim, embed_dim)
```

```
    def forward(self, query, key, value, mask=None):
        """
        Perform forward propagation of cross attention.
        Args:
            query (torch.Tensor): The current input of the decoder or the output of the previous layer.
                                  Shape: (batch_size, query_seq_len, embed_dim)
            key (torch.Tensor): The final output of the encoder.
                                Shape: (batch_size, key_value_seq_len, embed_dim)
            value (torch.Tensor): The final output of the encoder.
                                  Shape: (batch_size, key_value_seq_len, embed_dim) mask
            (torch.Tensor, optional): Attention mask (ignore padding tokens, etc.).
                                      Shape: (batch_size, 1, 1, key_value_seq_len) or (batch_size, 1, query_seq_len,
key_value_seq_len)
        Returns:
            tuple:
                - output (torch.Tensor): Output of the cross attention layer.
                                         Shape: (batch_size, query_seq_len, embed_dim) -
                attention_weights (torch.Tensor): The computed attention weights.
                                                  Shape: (batch_size, num_heads, query_seq_len, key_value_seq_len)
        """

        batch_size = query.size(0)


        # 1. Linear projection for Q, K, and V.
        # Q: (batch_size, query_seq_len, embed_dim) -> (batch_size, query_seq_len, embed_dim)
        # K, V: (batch_size, key_value_seq_len, embed_dim) -> (batch_size, key_value_seq_len, embed_dim)
        Q = self.query_proj(query)
        K = self.key_proj(key)
        V = self.value_proj(value)


        #2. Head splitting.
        # Q, K, V: (batch_size, seq_len, embed_dim) -> (batch_size, seq_len, num_heads, head_dim) # Change the
        dimensions to (batch_size, num_heads, seq_len, head_dim) for batch matrix multiplication.
        Q = Q.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1, 2)
        K = K.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1, 2)
        V = V.view(batch_size, -1, self.num_heads, self.head_dim).transpose(1, 2)
```

```
        #3. Scaled dot-product attention.
        # (Q @ K_T) / sqrt(head_dim) #
        (batch_size, num_heads, query_seq_len, head_dim) @ (batch_size, num_heads, head_dim,
key_value_seq_len)
        # -> (batch_size, num_heads, query_seq_len, key_value_seq_len) scores =
        torch.matmul(Q, K.transpose(-2, -1)) / (self.head_dim ** 0.5)

        # Apply if a mask is provided.
        if mask is not None:
            # The mask must be broadcastable as (batch_size, num_heads, query_seq_len, key_value_seq_len)
Do it.
            # Typically, the mask is of the form (batch_size, 1, 1, key_value_seq_len).
            scores = scores.masked_fill(mask == 0, float("-inf"))

        attention_weights = F.softmax(scores, dim=-1)

        # 4. Multiply by Value.
        # (batch_size, num_heads, query_seq_len, key_value_seq_len) @ (batch_size, num_heads,
key_value_seq_len, head_dim)
        # -> (batch_size, num_heads, query_seq_len, head_dim)
        weighted_values = torch.matmul(attention_weights, V)

        # 5. Connect the heads and perform the final linear projection.
        # (batch_size, num_heads, query_seq_len, head_dim) -> (batch_size, query_seq_len, num_heads,
head_dim)
        # -> (batch_size, query_seq_len, embed_dim)
        weighted_values = weighted_values.transpose(1, 2).contiguous().view(batch_size, -1, self.embed_dim)

        output = self.out_proj(weighted_values)

        return output, attention_weights # attention_weights can be returned for visualization, etc.

# --- Usage examples ---
if __name__ == "__main__":
```

```
# Example of encoder output (key, value) and decoder input (query).
batch_size = 2
encoder_seq_len = 10 # This is the length of the English
sentence decoder_seq_len = 5 # This is the current length of the
Korean translation embed_dim = 512
num_heads = 8

# The final output of the encoder (to be used as key and value). #
This is the embedding of the English sentence "I love machine learning."
encoder_output = torch.randn(batch_size, encoder_seq_len, embed_dim)

# The current input of the decoder or the output of the previous decoder layer (to be used as a query). #
This is the embedding of the Korean translation of "I do machine learning."
decoder_input = torch.randn(batch_size, decoder_seq_len, embed_dim)

# Initialize the cross attention layer.
cross_attention_layer = CrossAttention(embed_dim, num_heads)

# Example mask (ignoring padding tokens). # Assume the 5th
token of encoder_output is padding.
encoder_padding_mask = torch.ones(batch_size, 1, 1, encoder_seq_len, dtype=torch.bool) encoder_padding_mask[:, :, :,
4] = False # Set the position corresponding to the 5th token to False.

# Perform cross attention.
# Query: decoder_input (current status of Korean translation)
# Key, Value: encoder_output (English original)
output, attn_weights = cross_attention_layer(
    query=decoder_input,
    key=encoder_output,
    value=encoder_output,
    mask=encoder_padding_mask # Apply mask.
)

print("Input query (decoder input) shape:", decoder_input.shape) print("Input
key/value (encoder output) shape:", encoder_output.shape)
```

```
    print("Cross attention output shape:", output.shape)
    print("Attention weight shape (batch_size, num_heads, query_seq_len, key_value_seq_len):",
attn_weights.shape)

    # Output Shape: (batch_size, decoder_seq_len, embed_dim) #
    Attention weight Shape: (batch_size, num_heads, decoder_seq_len, encoder_seq_len)
```

4.5 Summary of Transformer Components

4.5.1 Scaled Dot Product Attention

This module is the core component that performs the basic operations of attention. It performs the following operations using the input vectors Q (Query), K (Key), and V (Value):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

• The inner product of Q and K indicates how related the current token is to other tokens in the context.

• Divide by sqrt{d_k} for normalization.

• Probabilize the weights through softmax and multiply them by V to generate the final attention output.

This module is implemented in PyTorch using torch.matmul, F.softmax, etc.

4.5.2 Multi-Head Attention

Because calculating attention as a single vector results in significant information loss, multiple attention "heads" are run in parallel and the results are then concatenated. Each head independently linearly transforms Q, K, and V and then applies Scaled Dot Product Attention.

This module includes the following steps:

• Separate Q, K, and V according to the number of heads,

• After performing attention for each head,

• Concatenate the results to create the final output.

4.5.3 Positional Encoding

Since the transformer does not consider the order, the position information of each word must be encoded.

For this purpose, a position encoding vector is generated based on the sine (sin) and cosine (cos) functions.

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

The encoding is added to the embedding vector and used as input.

4.5.4 Position-wise Feed Forward Network

This is an MLP structure that is applied independently to each token. Typically, a ReLU activation function is inserted between two linear layers. Its format is as follows:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

This serves to expand expressive power on a token-by-token basis.

4.5.5 Layer Normalization Residual Connection &

The transformer uses residual connection and layer normalization together in each module.

• Residual Connection: Reduces information loss by adding input as is.

• LayerNorm: Ensure learning stability through distribution normalization

These two factors help prevent performance degradation as the model depth increases.

4.6 Summary of Transformer Encoder Structure

The transformer encoder consists of the following sequence:

1. Add embedding + positional encoding
2. Multi-Head Attention
3. Residual + LayerNorm
4. Feed Forward Network

5. Residual + LayerNorm

This entire block is repeated several times to gradually enrich the contextual information of the sentence.

4.7 Analysis of core code based on PyTorch

The following is pseudocode that simplifies the structure by core module.

4.7.1 Scaled Dot Product Attention

```
class ScaledDotProductAttention(nn.Module):
    def forward(self, Q, K, V):
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k) attn
        = F.softmax(scores, dim=-1)
        return torch.matmul(attn, V)
```

4.7.2 MultiHeadAttention

```
class MultiHeadAttention(nn.Module):
    def forward(self, Q, K, V):
        # Linear transformation and split of Q, K,
        and V # Calculate attention for each head
        # Linear transformation of output after concat
        return output
```

4.7.3 Positional Encoding

```
class PositionalEncoding(nn.Module):
    def forward(self, x):
        # Calculate sin and cos vectors by location and add them to the input
        return x + self.pe[:, :x.size(1)]
```

4.7.4 EncoderLayer

```
class EncoderLayer(nn.Module):
    def forward(self, x):
        x = x + self_attn(x, x, x) # Residual x =
        LayerNorm(x) x =
        x + FFN(x) # Residual
        x = LayerNorm(x)
        return x
```

5. Multimodal OpenAI CLIP

5.1 CLIP Overview

CLIP (Contrastive Language–Image Pre-training) is a model developed by OpenAI that trains text and images together, demonstrating powerful performance across a variety of multimodal tasks. CLIP is trained by linking natural language descriptions to images, and can be used for a variety of tasks, including image classification, search, and caption generation.

5.2 Key Features

ÿ **Text-Image Matching:** Calculate the similarity between text descriptions and images.

ÿ **Zero-shot learning:** Applicable to new datasets without additional learning.

ÿ **Multimodal search:** Search images using text, or search text using images possible.

ÿ **Image classification:** Classification using natural language rather than existing labels.

5.3 Installation

CLIP operates on the PyTorch library in a Python environment. Installation is as follows:

5.3.1 How to install

pip install git+https://github.com/openai/CLIP.git

pip install torch torchvision

5.3.2 Dependencies

Python 3.7+

PyTorch 1.7.1+

torchvision

5.4 Loading the model

CLIP provides a variety of pre-trained models. The following code loads the models.

Example code:

```
import clip
```

```
import torch

# Loading the CLIP model and tokenizer
model, preprocess = clip.load("ViT-B/32", device="cuda" if torch.cuda.is_available() else
"cpu")
```

5.5 Calculating Image and Text Similarity

CLIP calculates similarity by transforming representations of images and text into the same embedding space.

5.5.1 Example code:

```
from PIL import Image

# Prepare images and text
image = preprocess(Image.open("example.jpg")).unsqueeze(0).to(device) text = clip.tokenize(["A
dog", "A cat"]).to(device)

# Model prediction
with torch.no_grad():
    image_features = model.encode_image(image) text_features
    = model.encode_text(text)

# Similarity calculation
logits_per_image, logits_per_text = model(image, text) probs =
logits_per_image.softmax(dim=-1).cpu().numpy()

print("Label probabilities:", probs)
```

5.6 Zero-shot image classification

CLIP can be used for various image classification tasks without additional training.

Example code:

```
# Defining classification labels
labels = ["a photo of a dog", "a photo of a cat"] text =
clip.tokenize(labels).to(device)
```

```
# Image processing
image = preprocess(Image.open("example.jpg")).unsqueeze(0).to(device)

# Prediction
with torch.no_grad():
    image_features = model.encode_image(image) text_features =
    model.encode_text(text)

    # Similarity calculation
    logits_per_image = (image_features @ text_features.T).softmax(dim=-1)

# Output results
print("Predicted label:", labels[logits_per_image.argmax()])
```

5.7 Multimodal Search

CLIP supports text-based image search and image-based text search.

Example code

```
# Data preparation
images = [preprocess(Image.open(path)).unsqueeze(0).to(device) for path in ["image1.jpg", "image2.jpg"]] texts = clip.tokenize(["A
beach", "A
forest"]).to(device)

# Image and text feature extraction
with torch.no_grad():
    image_features = torch.cat([model.encode_image(img) for img in images]) text_features =
    model.encode_text(texts)

# Similarity calculation
similarity = image_features @ text_features.T

print("Similarity matrix:", similarity.cpu().numpy())
```

5.8 Model Extension

CLIP is scalable for a variety of tasks and can be fine-tuned for use on custom datasets. However, caution is advised, as fine-tuning can degrade basic zero-shot performance.

Example code:

```
# Preparing a custom dataset
from torch.utils.data import DataLoader

custom_dataset = CustomDataset("path/to/data")
data_loader = DataLoader(custom_dataset, batch_size=32, shuffle=True)

# Model fine-tuning
optimizer = torch.optim.Adam(model.parameters(), lr=1e-5)
criterion = torch.nn.CrossEntropyLoss()

for epoch in range(epochs): for
    images, texts, labels in data_loader:
        images = preprocess(images).to(device)
        texts = clip.tokenize(texts).to(device)

        optimizer.zero_grad()

        image_features = model.encode_image(images)
        text_features = model.encode_text(texts)

        loss = criterion(image_features, labels)
        loss.backward()
        optimizer.step()
```

5.9 Conclusion

CLIP is a powerful model capable of integrating text and images, delivering superior performance across a variety of multimodal tasks. Leveraging this model, tasks previously difficult for existing deep learning models can now be easily accomplished.

6. Variational Autoencoders (VAE)

6.1 Introduction

Variational Autoencoders (VAEs) are probabilistic generative models used in deep learning for data compression and generation. They are an extended form of autoencoders, capable of representing data as a probability distribution in a latent space and utilizing this distribution to generate new data.

It learns by encoding input data into a latent space and then decoding it to reconstruct the original data. The goal of VAE is to extract the given data from the probability distribution of the latent space.
It is about sampling and generating new data through it.

A VAE consists of an encoder and a decoder. The encoder maps input data to a distribution in the latent space, and the decoder transforms this latent vector back into the original data. VAEs utilize variational inference to estimate and learn the distribution of latent variables.

6.2 Key Concepts of VAE

VAE has a similar structure to a general autoencoder, and compresses the input data and
It performs the process of restoration. However, the autoencoder simply reconstructs the input data.
While VAE focuses on learning probability distributions in latent space and exploiting them to generate new
It is designed to generate data.

Autoencoder

An autoencoder consists of an encoder, which compresses input data into a low-dimensional space, and a decoder, which restores the data back to its original state. It is used to efficiently compress input data and is trained to minimize the difference between the input and output.

Variational Autoencoder

VAEs introduce a probabilistic approach to overcome the limitations of autoencoders. The encoder transforms input data into a probability distribution represented by a mean (ÿ) and standard deviation (ÿ), and the decoder reconstructs the data based on values sampled from this distribution. This process is designed to probabilistically sample data from the latent space, enabling the generation of new data.

6.3 Structure of VAE

VAE consists of three main components: an encoder, a reparameterization trick, and a decoder.

Encoder: The
encoder maps input data to a probability distribution in the latent space. Specifically, it represents input data as a mean (ÿ) and standard deviation (ÿ), and through this, it learns to ensure that data in the latent space follows a normal distribution.

Reparameterization Trick is a technique used to enable
backpropagation during the process of sampling the latent variable zz. This is
The latent variable is expressed as follows:

$$z = \mu(x) + \sigma(x) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0,1)$$

This method enables probabilistic sampling.

The decoder
reconstructs the input data based on data sampled from the latent space. This process is trained to produce output that is as similar as possible to the original data.

6.4 VAE Learning Objectives

6.4.1 KL-divergence and VAE Role

In the learning process of VAE, the Kullback-Leibler Divergence (KL Divergence) plays a crucial role. KL divergence is a measure of the difference between two probability distributions, and in VAE, the goal is to minimize the difference between the latent distribution (q(z|x)) and the standard normal distribution (p(z)).
KL divergence is defined as follows:

D_{KL}(q(z|x) || p(z)) = ÿ q(z|x) log (q(z|x) / p(z)) dz In VAE, q(z|x) is
the conditional distribution of the latent variable z with respect to the input x, which is expressed as a normal distribution N(ÿ, ÿ²) with mean ÿ and variance ÿ². On the other hand, p(z) is a standard normal distribution N(0, 1). VAE is trained to reduce the difference between these two distributions.

6.4.2 Mathematical Expansion of KL Divergence

In the process of calculating the KL-divergence, since both q(z|x) and p(z) are normal distributions, the KL-divergence between these two distributions is simply developed as follows.

The normal distribution q(z|x) has mean ÿ and variance ÿ², and the standard normal distribution p(z) has mean 0 and variance 1.

If we calculate the KL divergence, it expands as follows:

D_{KL}(q(z|x) || p(z)) = ÿ N(ÿ, ÿ²) log (N(ÿ, ÿ²) / N(0, 1)) dz

As a result, the KL divergence can be expressed in a simple formula as follows:

D_{KL}(q(z|x) || p(z)) = 0.5 [log(1/ÿ²) + ÿ² + ÿ² - 1]

6.4.3 KL-divergence item-by-item interpretation

Each item in the KL divergence has the following meaning:

1. log(1/ÿ²): Measures the amount of information relative to the variance ÿ². A larger ÿ² indicates a wider distribution, meaning less dense information.

2. ÿ²: How close or far is the variance of q(z|x) from the variance of 1 (standard normal distribution)?
Measures whether it exists.

3. ÿ²: Measures how much the mean of q(z|x) differs from the mean (0) of the standard normal distribution. ÿ²
The larger it is, the further the mean of q(z|x) is from the standard normal distribution.

4. -1: Since the variance of the standard normal distribution is 1, the last term of the KL divergence is a correction for this value.

6.4.4 Learning process of VAE and KL divergence role

VAE optimizes by combining reconstruction loss and KL divergence as loss functions.

The function is as follows:

L = E_{q(z|x)}[log p(x|z)] - D_{KL}(q(z|x) || p(z))

The first term, E_{q(z|x)}[log p(x|z)], is the reconstruction error, which is how well the model reconstructs the input.

The second item, D_{KL}(q(z|x) || p(z)), is the KL divergence, which means that the distribution of the latent space is standard
Measures how close it is to a normal distribution.

VAE is trained by minimizing these two terms simultaneously. The KL-divergence term is a term that is distributed in the latent space as a standard
It is induced to approach a normal distribution, and the reconstruction error is adjusted to ensure that the input data can be reconstructed well.
Induce.

6.4.5 Pseudocode for VAE training

The following is the pseudocode for calculating KL divergence during the VAE training process and reflecting it in the loss function.
We can write together.

```
import torch
import torch.nn as nn

# Define the VAE model (encoder, decoder)
class VAE(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(input_dim, latent_dim) # Encoder self.decoder =
        Decoder(latent_dim, input_dim) # Decoder

    def forward(self, x): #
        encoding
        mu, log_var = self.encoder(x) z =
        self.reparameterize(mu, log_var)

        # Decoding
        reconstructed_x = self.decoder(z)
        return reconstructed_x, mu, log_var

    def reparameterize(self, mu, log_var): #
        reparameterization trick std =
        torch.exp(0.5 * log_var) eps =
        torch.randn_like(std)
        return mu + eps          * std

# KL-divergence calculation function
def kl_divergence(mu, log_var): # KL-
    divergence: 0.5 * (mu^2 + exp(log_var) - log(var) - 1) return -0.5 *
    torch.sum(1 + log_var - mu.pow(2) - log_var.exp())

# Loss function
def loss_function(reconstructed_x, x, mu, log_var):
    # Reconstruction loss
    reconstruction_loss = nn.MSELoss(reduction='sum')(reconstructed_x, x)
```

```
        # KL-divergence loss
        kl_loss = kl_divergence(mu, log_var)

        # Total loss
        return reconstruction_loss + kl_loss

# Model training example
vae = VAE(input_dim=784, latent_dim=128)
optimizer = torch.optim.Adam(vae.parameters(), lr=1e-3)

for epoch in range(epochs): for
    batch_idx, (data, _) in enumerate(train_loader):
        optimizer.zero_grad()

        # Forward propagation
        reconstructed_x, mu, log_var = vae(data)

        # Loss calculation
        loss = loss_function(reconstructed_x, data, mu, log_var)

        # Backpropagation
        loss.backward()
        optimizer.step()
```

6.5 Key Features of VAE

VAE has the following characteristics:

Generative

models can generate new data by sampling from the latent space, a key feature that conventional autoencoders lack.

It learns low-

dimensional representations of **latent space learning** data, and can use these representations to analyze or transform the characteristics of the data.

Probabilistic

Approach Probabilistic approach allows the model to learn and generalize from data more flexibly.

6.6 Applications of VAE

VAEs are being used in a variety of fields, including:

Image

generation is used to generate new images from datasets such as MNIST and CIFAR-10.

Data clustering can

analyze clusters of data in a latent space or identify differences between clusters.

Noise removal

is used to remove noise from damaged data or to restore defective data.

Time series data

analysis is used to transform time series data or predict future data.

6.7 Conclusion

VAE not only compresses data efficiently through a probabilistic approach, but also
It is a powerful generative model that can generate various data analysis and generation applications.
VAE has become an important tool in the field. Future development of VAE will lead to more complex data.
It is expected to contribute to understanding and utilizing the structure.

7. Stable Diffusion

7.1 Overview

Stable Diffusion, an innovative generative model that generates high-quality images based on text descriptions, has recently attracted significant attention. This model utilizes a diffusion model to incrementally generate images, demonstrating outstanding performance in the text-to-image conversion process. Stable Diffusion is open-source and available for use by a wide range of researchers and developers.

Stable Diffusion is a generative model within the Latent Diffusion Model (LDM) family that generates high-resolution images based on input text prompts. Rather than performing diffusion directly in the high-resolution image space, this model efficiently removes noise in the latent representation space of the image. Stable Diffusion consists of three main components:

1. VAE (Variational Autoencoder): Image ÿ latent space transformation

2. U-Net: A denoising network that restores noisy latents.

3. Text Encoder (CLIP): Embed text and use it as a condition.

The inference process starts with “complete noise” and progressively removes noise to produce images that match the text conditions.

7.2 Overview of the Diffusion Model

The diffusion model is a method for generating images by **adding and removing noise** during the image generation process. This process is broadly divided into two parts:

Forward Process: This process involves gradually adding noise to the original image, ultimately creating a completely noisy image. This process gradually removes detail from the image.

Denoising (Reverse Process): This process uses a trained model to restore a noisy image to its original form. The model removes noise through the image restoration process, ultimately producing a high-quality image.

7.3 How Stable Diffusion Works

The working principle of Stable Diffusion is divided **into three main components** required to generate an image based on a text description :

Noise Addition and Removal Process: Stable Diffusion extends the principles of the diffusion model. The model uses an improved version of **the Denoising Diffusion Probabilistic Models (DDPM)** method to progressively generate images, gradually recovering the information needed to generate images from text.

Text embedding: This is the process of converting text descriptions into vector form. Stable Diffusion effectively embeds text using **the CLIP model .** The CLIP model is a powerful tool for linking text and images, allowing it to generate images that accurately reflect text descriptions.

U-Net Architecture: Stable Diffusion utilizes the U-Net architecture in image generation.

The architecture effectively utilizes information through downsampling and upsampling.

It transmits and efficiently performs the image restoration process.

7.4 Learning Process of Stable Diffusion

Stable Diffusion is trained on a large-scale **text-image dataset .** This dataset is

It consists of an image and its corresponding text description, and the model converts the text description into an embedding vector.

Then, we learn how to generate high-quality images through the process of removing noise.

In the process, the model learns the relationship between text and images, and based on this, it learns various texts.

It can generate appropriate images for the input.

7.5 Text-to-Image Mapping

Stable Diffusion uses **the CLIP model** to convert text into images . CLIP is a model that converts text into images.

A model that can process images simultaneously and learns to match text descriptions with images.

Stable Diffusion utilizes this model to accurately identify the relationship between text and image and generate

an image appropriate for the text. In this process, an image that accurately reflects the meaning of the text is generated.

It is created incrementally.

7.6 Stable Diffusion Operation Process

The operation of Stable Diffusion is broadly as follows.

7.6.1 Summary of the entire operation process

Embed the input text.

Set the starting point as noise in latent space.

Gradually remove noise using U-Net.

The final latent is decoded through VAE to restore the image.

7.6.2 Main Full process pseudocode

```
#1. Text embedding
text_embedding = TextEncoder(prompt)

#2. Generate initial latent noise
```

```
latent = sample_normal_distribution(shape=(batch_size, latent_dim))

#3. Repeat noise removal
for timestep in reverse(timesteps):
    noise_pred = UNet(latent, timestep, text_embedding) latent =
    update_latent(latent, noise_pred, timestep)

#4. Restore the image by decoding the latent
image = VAE_Decoder(latent)
```

7.6.3 VAE (Variational Autoencoder)

VAE encodes **images** into **latent** space and restores the latent space back **to an image .**
In Stable Diffusion, we avoid high-resolution direct processing and perform operations in a small latent space (e.g., 64x64x4).

Main action pseudocode:

```
# Encoder: image ÿ latent
mu, sigma = Encoder(image) z = mu
+ sigma * random_normal_like(mu) # Use reparameterization trick

# Decoder: latent ÿ image
reconstructed_image = Decoder(z)
```

explanation:

• **The encoder** compresses the input image to generate ÿ(x), ÿ(x). • The latent zz is sampled using **the reparameterization trick . • The decoder** receives the latent zz and restores the image.

7.6.4 U-Net

U-Net is a network that receives **noisy latent**
input and removes noise by predicting "what kind of noise is mixed in."

In Stable Diffusion, a **cross-attention** structure is added to the general U-Net to integrate text conditions into the image restoration process.

7.6.5 Main action pseudocode

```
def UNet(latent, timestep, text_embedding):
    x = initial_conv(latent)

    for down_block in downsampling_path:
        x = down_block(x)
        x = cross_attention(x, text_embedding)

    x = bottleneck(x)

    for up_block in upsampling_path:
        x = up_block(x) x
        = cross_attention(x, text_embedding)

    output = final_conv(x)
    return output # Predicted noise
```

7.6.6 explanation

• Features are extracted and restored through the downsampling ÿ upsampling
path. • **Cross-Attention** is performed in each main block to reflect text information. • Finally,
the noise mixed in the current latent is predicted and returned.

7.6.7 Text Encoder (CLIP Text Encoder)

The Text Encoder takes a text prompt and converts it into a fixed-size embedding vector usable by the Attention module of U-Net.

In Stable Diffusion, we use a modified version of OpenAI CLIP's Text Encoder.

Main action pseudocode

```
def TextEncoder(prompt):
    tokens = tokenize(prompt)
    text_embedding = transformer_encoder(tokens) return
    text_embedding
```

explanation

• First, tokenize the input text. • Obtain an embedding for each token through the Transformer. • Finally, this text embedding is fed into the Attention module of U-Net.

7.6.8 Final Summary

Stable Diffusion operates on the following structure:

1. Embed the text. 2. Start with random noise latent.
3. Gradually remove noise and refine latent data through U-Net.
4. The final latent is restored into a high-resolution image through VAE.

This process is learned in the forward direction (noise addition ÿ noise removal prediction) during learning, and in the reverse direction (noise removal ÿ image restoration) during generation.

7.7 How to Install and Run Stable Diffusion

To run Stable Diffusion, you must first install **the required libraries and dependencies .** Here's the installation process:

System Requirements:

Python 3.8 or later

PyTorch 1.7 or higher

CUDA 11.1 or higher (when using GPU)

Clone the repository: Clone the source for Stable Diffusion from GitHub:

git clone https://github.com/Stability-AI/stablediffusion

cd stablediffusion

Install required packages: Install the required Python libraries:

pip install -r requirements.txt

Download the model file: The model file can be found on the official page of Stable Diffusion. You can download it.

Running the model: To run the model, use the following command:

python scripts/txt2img.py --prompt "a beautiful landscape" --plms

This command creates an image that matches the text description.

7.8 Utilizing Stable Diffusion

Stable Diffusion is widely used primarily in **artistic** work and **digital** art. It can automatically generate high-quality images using text prompts, saving designers and artists significant time during the creative process. It can also be utilized in a variety of industries , including **game** development, **advertising design, and fashion design .**

7.9 Advantages of Stable Diffusion

The greatest advantages of Stable Diffusion are **its openness and** efficiency. Compared to existing GAN models, it is more stable and can perform text-to-image conversion very effectively. Furthermore, its open-source nature makes it extremely useful, allowing anyone to leverage it for a variety of projects.

7.10 Conclusion

Stable Diffusion is a model that represents a significant advancement in image generation technology. It utilizes diffusion models to reliably generate high-quality images, and the CLIP model is highly effective in accurately mapping text to images.

7.11 References

Stable Diffusion GitHub: https://github.com/Stability-AI/stablediffusion

Link provided on the blog: https://daddynkidsmakers.blogspot.com/2024/02/stable- diffusion.html

8. Full fine-tuning (FFT)-based model learning

8.1 Small Language Model (sLLM)

sLLM (Small Language Model) refers to a small natural language processing model with a relatively small number of parameters and capable of running even in environments with limited computational resources. GPT-2 small, DistilBERT, TinyLLaMA, and Phi-2 are representative examples of sLLMs. sLLMs are lightweight and can operate on IoT, edge devices, and mobile devices, and they also excel in response speed and inference cost.

8.2 Full Fine-Tuning (FFT)

FFT is a method that retrains all parameters of a pretrained language model. By updating all weights of an existing LLM, a model fully optimized for a specific domain can be created.

merit:

Maximize domain-specific performance

Free model customization possible

disadvantage:

It requires a lot of resources such as memory and GPU.

Example) Large models such as LLaMA 7B and above: Requires at least 24–80 GB (depending on optimization)

Long learning time and high cost

Concerns about overfitting exist

8.3 PEFT(Parameter-Efficient Fine-Tuning)

PEFT is a lightweight learning technique that fine-tunes only a few parameters (modules) rather than the entire model. Representative examples include LoRA, Prefix Tuning, Adapter Tuning, and BitFit.

merit:

Resource-efficient due to small number of learning parameters

Preserve the original weights of the existing model

Fast learning and high reusability

disadvantage:

Limitations on extreme domain transitions

Performance may be limited depending on the quality of the original model.

8.4 Distributed Learning Strategies: DP, DDP, FSDP, DeepSpeed

8.4.1 Data Parallelism (DP)

It is a structure that copies the same model on multiple GPUs and distributes training on different batches of data.

Implemented using PyTorch's DataParallel API, if one GPU lags behind the others during training, the overall speed slows down. Furthermore, communication to the main GPU at each step for parameter synchronization becomes a bottleneck.

```
import torch

import torch.nn as nn



model = MyModel()

model = nn.DataParallel(model) # Low scalability compared to single GPU memory
```

8.5 DistributedDataParallel (DDP)

DDP is PyTorch's official distributed parallel processing technique, where each GPU independently computes parameters and communicates during the backward pass to average the gradients. DDP is efficient and allows GPU utilization across nodes.

```
# DDP example (single node, multi-GPU)

import os

import torch

import torch. distributed as dist

from torch.nn.parallel import DistributedDataParallel as DDP



os.environ['MASTER_ADDR'] = 'localhost'

os.environ['MASTER_PORT'] = '12355'

dist.init_process_group(backend='nccl', rank=0, world_size=1)
```

```
model = MyModel().cuda()

model = DDP(model)
```

8.6 Fully Sharded Data Parallel (FSDP)

This technique reduces memory usage by distributing model parameters, gradients, and optimizer states across GPUs. Even very large models can be trained with minimal memory. Since each GPU only holds a portion of the entire model, it eliminates memory bottlenecks.

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP

from torch.distributed.fsdp.wrap import default_auto_wrap_policy


model = MyModel().cuda()

fsdp_model = FSDP(model, auto_wrap_policy=default_auto_wrap_policy)
```

8.7 DeepSpeed

DeepSpeed is a distributed learning framework developed by Microsoft. It utilizes the ZeRO (Zero Redundancy Optimizer) algorithm to distribute and store optimizer state, gradients, and parameters. It also supports memory savings and speed improvements through offloading, mixed precision, pipeline parallelism, and activation checkpointing.

Example usage:

Create a deepspeed_config.json configuration file (specify learning stage, ZeRO stage, batch size, etc.)

Run the model with the following code:

```
import deepspeed

model = MyModel()

model_engine, optimizer, _, _ = deepspeed.initialize(

    model=model,

    config="deepspeed_config.json",

    model_parameters=model.parameters()
```

```
)
```

Example configuration file (deepspeed_config.json):

```
{
  "train_batch_size": 32,
  "fp16": {"enabled": true},
  "zero_optimization": {
    "stage": 2,
    "allgather_partitions": true,
    "allgather_bucket_size": 2e8,
    "overlap_comm": true,
    "reduce_scatter": true
  }
}
```

8.8 Conclusion

sLLM is a lightweight language model suitable for a variety of environments. For efficient fine-tuning, the PEFT technique is primarily used, rather than the FFT. For large-scale training, distributed learning frameworks such as DDP, FSDP, and DeepSpeed should be utilized to efficiently utilize resources.

9. PEFT-based Gemma-2B medical question-answer fine-tuning

9.1 Introduction

Large Language Models (LLMs) require massive computational resources, and fine-tuning all parameters incurs significant memory usage and computational costs. Consequently, Parameter-Efficient Fine-Tuning (PEFT) is gaining attention as an efficient, domain-specific fine-tuning method. This report presents a theoretical overview of PEFT, including Low-Rank Adaptation (LoRA), a representative PEFT technique, and analyzes the actual code used to fine-tune Hugging Face's Gemma-2B model for a medical question-answering task.

9.2 PEFT(Parameter-Efficient Fine-Tuning)

PEFT is an approach that enables efficient fine-tuning by targeting only specific modules or layers for learning without adjusting all the weights of a pre-trained large-scale language model.

The advantages are as follows:

Resource saving: Training only a portion of the model rather than the entire model reduces GPU memory usage.

Fast convergence: Converges quickly because there are few learning targets, and is effective even with small amounts of data.

Modularity and reuse: Multiple domain-specific LoRA adapters can be stored and reused for the same model.

Representative PEFT techniques include:

LoRA: Complementing linear operations by inserting low-rank matrices.

Adapter Tuning: Training by adding a small network to the Transformer block.

Prefix/Prompt Tuning: Insert domain-specific tokens into the input sequence.

9.3 LoRA (Low-Rank Adaptation) Theory and Application Structure

LoRA has fixed original weights for linear layers of self-attention or feedforward blocks.

By adding low-rank matrices, the number of parameters is reduced and efficient learning is possible.

Existing operations:

$$y = Wx \text{ LoRA}$$

After application:

$$y = (W + BA)x$$

Here, A and B are learnable matrices.

$$A \in \mathbb{R}^{r \times d}, B \in \mathbb{R}^{d \times r}$$

The r value is a key hyperparameter that controls learning ability and the number of parameters.

Typically, r is set between 8 and 64.

```
class LoRALayer(torch.nn.Module):
    def __init__(self, in_dim, out_dim, rank, alpha):
        super().__init__()
        std_dev = 1 / torch.sqrt(torch.tensor(rank).float())
        self.A = torch.nn.Parameter(torch.randn(in_dim, rank) * std_dev)
        self.B = torch.nn.Parameter(torch.zeros(rank, out_dim))
        self.alpha = alpha

    def forward(self, x):
        x = self.alpha * (x @ self.A @ self.B)
        return x
```

LoRAConfig parameter description:

```
LoraConfig(
    r=64,
    lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
lora_dropout=0.05,
```

```
    bias="none",
    task_type="CAUSAL_LM"
)
```

r: dimension of low-rank matrix, smaller size reduces computational load, larger size increases expressive power

lora_alpha: This acts as a scaling factor during training and is related to training stability.

target_modules: The name of the linear layer within the Transformer where LoRA will be inserted.

lora_dropout: Dropout probability applied during training

bias: Whether to train the bias term together with LoRA (none, all, lora_only, etc.)

task_type: The type of fine-tuning task (CAUSAL_LM, SEQ_CLS, etc.). CAUSAL_LM: A language generation model that sequentially predicts the next token (e.g., GPT).

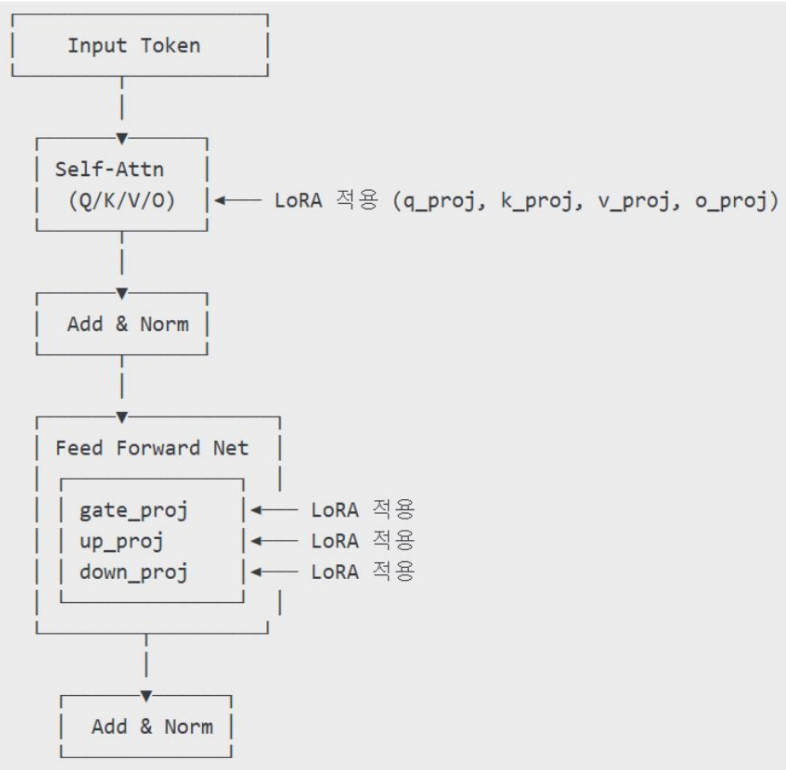
SEQ_CLS: Classification models that predict a single class for the entire input (e.g., sentiment analysis, sentence classification)

Applicable modules:

q_proj, k_proj, v_proj, o_proj: Correspond to the query, key, value, and output projections of Self-Attention.

gate_proj, up_proj, down_proj: The three core linear layers used in the Feedforward Network (FFN) of the Transformer block.

- gate_proj: Processes input before applying the gating mechanism
- up_proj: Expands the hidden dimension (ex. 4096 → 11008)
- down_proj: Reduce the dimension again and restore the original output



LoRA is inserted by appending learnable low-rank matrices to these projection layers, and during the actual forward calculation, the original weights are fixed and fine-tuned only through the BA matrix.

9.4 Quantization and BitsAndBytesConfig Description

Quantization is a technique that reduces resource usage by representing model parameters as int4/8 instead of float. bitsandbytes is the quantization library adopted by Hugging Face, and is particularly effective in 4-bit quantization.

Example settings:

```
BitsAndBytesConfig(
  load_in_4bit=True,
  bnb_4bit_use_double_quant=True;
  bnb_4bit_quant_type="nf4",
  bnb_4bit_compute_dtype=torch.bfloat16
)
```

Option Description:

• load_in_4bit: Whether to load the model in 4 bit •

bnb_4bit_use_double_quant: Improve expressiveness by performing double quantization

• bnb_4bit_quant_type:

o nf4: NormalFloat4, normalized 4-bit floating point o fp4: Basic

floating point 4-bit quantization, low precision • bnb_4bit_compute_dtype:

Specifies the compute dtype (bfloat16, float16, float32) . Typically

Using bfloat16

9.5 Dataset Structure and Prompt Templates

```
{
  "instruction": "problem",
  "input": "(optional) additional information",
  "output": "Correct answer"
}
```

Prompt configuration:

<start_of_turn>user

Below is an instruction that describes a task. Write a response that appropriately completes the request.

{instruction + input}

<end_of_turn>

<start_of_turn>model

{output}

<end_of_turn>

Recommended training data size:

1,000~10,000 units ÿ Suitable for small-scale LoRA fine-tuning

10,000 or more ÿ Stable convergence and generalization performance can be achieved.

9.6 Detailed description of the entire fine-tuning code

9.6.1 Model and quantization settings

bnb_config = BitsAndBytesConfig(...)

model = AutoModelForCausalLM.from_pretrained(model_id,
quantization_config=bnb_config, device_map="auto")

Load pre-trained models in 4-bit format and automatically assign them to GPUs.

9.6.2 Preparing the Tokenizer

tokenizer = AutoTokenizer.from_pretrained(model_id, add_eos_token=True)

tokenizer.pad_token = tokenizer.eos_token

tokenizer.padding_side = 'right'

9.6.3 Dataset Processing

text_column = [generate_prompt(dp) for dp in dataset['train']]

dataset = dataset['train'].add_column("prompt", text_column)

dataset = dataset.map(lambda samples: tokenizer(samples["prompt"]), batched=True)

Combine instruction and input to generate a prompt and then convert it to a tokenizer.

9.6.4 Data Separation

dataset = dataset.train_test_split(test_size=0.1)

train_data = dataset["train"]

test_data = dataset["test"]

9.6.5 LoRA Set up and apply

model = prepare_model_for_kbit_training(model)

lora_config = LoraConfig(...)

model = get_peft_model(model, lora_config)

Activate only the learnable part and apply the LoRA adapter

9.6.6 **Prepare and perform the study:**

```
trainer = SFTTrainer(
    model=model,
    train_dataset=train_data,
    eval_dataset=test_data,
    peft_config=lora_config,
    args=transformers.TrainingArguments(
        per_device_train_batch_size=1;
        gradient_accumulation_steps=4;
        max_steps=100,
        learning_rate=2e-4,
        output_dir="outputs",
        save_strategy="epoch"
    ),
    data_collator=transformers.DataCollatorForLanguageModeling(tokenizer, mlm=False);
)
trainer.train()
```

Supervised Fine-Tuning is a Trainer that supervises pre-trained LLM with (prompt, answer) pair data as follows.

input (prompt)	**output (correct answer)**
"Where is Seoul?" "Seoul is in South Korea."	

ÿ per_device_train_batch_size: Sets the mini-batch size to be processed per GPU. Memory If less, set to 1.

ÿ gradient_accumulation_steps: Accumulate gradients for multiple steps and then apply them only once.
Perform backward/update. Effective when VRAM is insufficient, the above settings are batch size=4 and
Similar effect.

ÿ max_steps: The number of optimizer steps. The learning termination condition based on the number of steps.

ÿ learning_rate: Optimizer learning rate. In LoRA, 2e-4 to 5e-4 is a typical range.

ÿ output_dir: Directory path to save learning results, checkpoints, logs, etc.

ÿ save_strategy: Defines the model saving cycle. If it is epoch, it saves every epoch.

9.6.7 Merge and save models

```
merged_model = PeftModel.from_pretrained(base_model, new_model).merge_and_unload()
merged_model.save_pretrained("merged_model")
tokenizer.save_pretrained("merged_model")
```

Merge the final model with the base model and save it in a format suitable for inference.

9.7 References

https://lightning.ai/lightning-ai/studios/code-lora-from-scratch?section=featured

9.8 Conclusion

Applying LoRA-based PEFT to Gemma-2B model to perform question-answering learning in medical field
We covered the method. We analyzed the structure of LoRA, the adapter insertion location, and the effect of rank
setting, and explained how 4-bit quantization using bitsandbytes saves resources. In addition,
As an example of a user dataset, the structure of a medical dataset, the appropriate training data size, and the entire training code
The flow was specifically organized.

10. Rama 3 Fine Tuning

This paper presents a local fine-tuning workflow based on the Meta LLaMA 3 model. Specifically, it presents a practical approach for fine-tuning large language models even in low-end environments, utilizing model quantization using QLoRA and LoRA-based parameter-efficient fine-tuning (PEFT).

Based on HuggingFace's learning pipeline, we will explain the entire process step-by-step, from data preprocessing to learning, evaluation, and deployment, using a real-world medical domain dataset as an example.

10.1 Description and loading of major libraries

import os, pandas as pd, json, torch, wandb

from transformers import AutoTokenizer, AutoModelForCausalLM

from huggingface_hub import login

from trl import SFTTrainer, setup_chat_format

from datasets import Dataset, load_dataset

ÿ Description of core libraries

transformers: Loads various pre-trained language models, training pipelines, and tokenizers.

Provide functionality.

wandb: Weights & Biases improves performance during repeated experiments through experiment tracking and visualization.

trl: A transformer-based RLHF and SFT toolkit that enables more structured LLM learning.

datasets: A high-performance dataset processing tool that enables efficient loading and parallel processing of large text datasets.

huggingface_hub: An API for storing and sharing models and tokenizers. Used to upload or download trained results.

10.2 Custom Data Loading Function: load_dataset_from_folder()

def load_dataset_from_folder(folder_path):

ÿ Function Description

Read JSON files existing in the local directory and process them into a structure that can be used for learning.

Merge the answer field in list form into a single string, based on a JSON structure with a key called responses.

Convert to a dataframe and wrap it in a HuggingFace Dataset object to integrate it into the subsequent training pipeline.

10.3 The entire fine-tuning pipeline: train()

Start learning according to your settings.

10.4 Base Model Setup and Authentication

```
base_model = "Undi95/Meta-Llama-3-8B-hf"

login(token='...')

wandb.login(key='...')
```

Load a LLaMA 3-based model from HuggingFace Hub. The current model is a GPT-style Causal Language Model with 8B parameters.

Set up access to external services using the HuggingFace token and wandb API key.

Experiment logs can be viewed in real time on the wandb dashboard, and are also useful for comparing experiments.

10.5 Loading and Shuffling User Datasets

```
custom_dataset = load_dataset_from_folder('./dataset')

custom_dataset = custom_dataset.shuffle(seed=65)
```

The custom dataset consists of local JSON files, and the learning order is randomized through shuffle(seed=65).

Seed fixation ensures reproducibility of results, meeting academic/industrial experimental standards.

10.6 QLoRA-based 4-bit quantization setup

bnb_config = BitsAndBytesConfig(...)

ÿ QLoRA Concept and Parameter Explanation

QLoRA stands for Q-formatted LoRA, which combines the LoRA technique with a quantized model.

load_in_4bit=True: Loading models with 4-bit precision reduces GPU memory usage by approximately 3-4x.

bnb_4bit_quant_type="nf4": NormalFloat4 is an improved format for expressing normal distributions with high learnability.

bnb_4bit_use_double_quant=True: Minimizes information loss and improves compression efficiency through secondary quantization.

bnb_4bit_compute_dtype=torch.float16: Operations are performed with float16 to ensure operational performance.

10.7 Loading Models and Tokenizers + Configuring Conversation Formats

model = AutoModelForCausalLM.from_pretrained(...)

tokenizer = AutoTokenizer.from_pretrained(...)

model, tokenizer = setup_chat_format(model, tokenizer)

ÿ **Description**

AutoModelForCausalLM stands for GPT-style token order prediction model, and LLaMA stands for
It applies.

setup_chat_format() applies a conversational prompt template to the model and tokenizer, enabling a consistent prompt structure for training and inference.

Especially when using role-based templates, role distinctions such as “user” and “assistant” have a significant impact on learning performance.

10.8 Efficient Fine-tuning of LoRA-Based Parameters

peft_config = LoraConfig(...)

model = get_peft_model(model, peft_config)

ÿ LoRA Core Elements

r=16: LoRA rank value. The smaller the value, the fewer parameters there are, making it more memory efficient.

lora_alpha=32: A scaling coefficient for adjusting the model's expressiveness.

target_modules: Set only attention projection layers such as q_proj and v_proj as learning targets.

ÿ Benefits

Since only a few percent of the total model parameters are learned, memory usage, training time, and storage space are significantly reduced.

10.9 Dataset Preprocessing and Integration

Preprocess the HuggingFace dataset (ai-medical-chatbot) and custom dataset into chat templates.

Convert user questions and responses into text in the role: user, role: assistant structure to create a unified learning structure.

After merging with concatenate_datasets(), split the training/validation data in a 9:1 ratio using train_test_split(test_size=0.1).

10.10 Configuring Learning Hyperparameters

training_arguments = TrainingArguments(...)

per_device_train_batch_size=1: Use small batches because 4-bit quantization requires more than 16GB of VRAM.

gradient_accumulation_steps=2: This has the effect of doubling the actual batch size, which induces stable learning.

optim="paged_adamw_32bit": Use the paged optimizer for memory optimization.

eval_steps=0.2: Performs validation at 20% intervals during epochs, enabling fast anomaly detection.

group_by_length=True: A technique to increase learning efficiency by grouping samples of similar length.

10.11 Supervised Learning-Based Training: SFTTrainer

```
trainer = SFTTrainer(...)

trainer.train()
```

SFTTrainer extends HuggingFace Trainer and is designed for text fine-tuning in LLM.

Setting packing=False means that each sample is trained as an independent sentence, which increases the stability of training.

Internally, it includes functions such as gradient clipping and learning rate scheduling.

10.12 Evaluation and Model Deployment

Qualitatively verify performance improvements by comparing the model's responses to the same questions before and after fine-tuning.

You can save it locally with model.save_pretrained() and upload it to HuggingFace Hub with push_to_hub().

The trained model can be easily utilized later using frameworks such as Inference API, Gradio, and FastAPI.

10.13 Conclusion and Extension

This course provides practical techniques for effectively fine-tuning LLaMA 3-based models in a low-cost environment. You'll also learn how to integrate cutting-edge research trends (e.g., QLoRA, PEFT, W&B logging) into your actual code. You can also consider expanding to advanced modeling such as vision-language models, instruction tuning, and Reinforcement Learning with Human Feedback (RLHF).

11. RAG (Retrieval-Augmented Generation)

11.1 Overview

Retrieval-Augmented Generation (RAG) is a method that overcomes the limitations of large language models (LLMs) by retrieving external knowledge bases and incorporating them into the generation process. RAG involves three steps: retrieving relevant information from external documents or databases, augmenting this information with the input, and ultimately generating text.

While existing LLMs generate responses using only information inherent in the parameters, RAG improves recency, accuracy, and factuality by utilizing retrieved external information.

11.2 History

RAG was featured in Facebook AI Research (FAIR) in 2020. It was first proposed by. This technique was developed with the aim of overcoming the limitations of LLM in open-domain question answering and fact-based generation, and it uses DPR (Dense Passage Retrieval). It was a structure that used a search engine and a pre-trained language model such as BART as a generator.

Learning follows a metric learning approach, constructing sets of correct and incorrect documents for a given question and optimizing a softmax-based loss that maximizes the similarity of correct documents and minimizes the similarity of incorrect documents:

DPR is an independent BERT for questions and documents. Embedding is performed through encoders (EQ, EP) and the dot product of the question vector and document vector is used as a similarity function. This method has the advantage of efficient similarity calculation and document embedding pre-indexing, and is compatible with FAISS (Facebook AI Similarity Search) . Quickly compare similarity across billions of documents using the same high-speed similarity search library. Can be.

Since then, the RAG concept has been implemented in various open source frameworks (HuggingFace Transformers, LangChain, Haystack, etc.), and when combined with vector DB, practical application cases have expanded to include PDF document question-and-answer, technical search, and legal document summarization.

11.3 Components

11.3.1 Retrieval and Indexing

11.3.1.1 outline

The process of searching external documents that are highly relevant to the user's question. Usually vector

Databases (e.g. FAISS, Chroma, Pinecone, etc.) and embedding models (OpenAI, HuggingFace, etc.)

It utilizes the following representative search algorithms:

11.3.1.2 Cosine Similarity Top-K Base search

This is the most basic vector similarity-based search.

By calculating the cosine similarity between the document embedding and the question embedding, K documents with high similarity are selected.

Choose.

Example of a problem: When a user asks, "What are the advantages of AI?", the same thing is repeated or very

Only similar documents may be selected, resulting in a lack of diversity. For example, "Improving the efficiency of AI"

Documents that repeat the same topic are selected repeatedly.

Development Background: High retrieval quality but low diversity negatively affects the response quality of subsequent generation models.

I can give it to you.

Pseudocode:

Input: query_embedding, document_embeddings, K

scores = [cosine_similarity(query_embedding, doc_emb) for doc_emb in document_embeddings]

ranked_docs = sort_by_score_descending(document_embeddings, scores)

Return top K ranked_docs

11.3.1.3 MMR (Maximal Marginal Relevance)

It was introduced to solve the problems of redundancy and lack of diversity of Cosine Top-K.

Example of a problem: When a user asks the question "What are the limits of autonomous driving?", only the existing cosine similarity

Using this feature will only continuously select documents related to "sensor accuracy issues." This will miss other perspectives

(legal, ethical, and technical).

Solution: Select documents in a way that reduces diversity with already selected documents while maintaining relevance to the question.

The importance of relevance and redundancy can be controlled through the ÿ value (0 ÿ ÿ ÿ 1). For example, ÿ=0.9 focuses on relevance, while ÿ=0.3 focuses on diversity.

ÿ=1.0: Same as the original cosine similarity, but does not consider redundancy.

ÿ=0.0: Consider only diversity (ignore relevance to the question).

ÿ=0.5: Considered balanced.

MMR pseudocode:

```
Input: query_embedding, candidate_docs, ÿ (0 <= ÿ <= 1), K
Initialize selected_docs = []
While len(selected_docs) < K:
    best_score = -inf
    best_doc = None
    For doc in candidate_docs:
        relevance = cosine_similarity(query_embedding, doc.embedding)
        diversity = max([cosine_similarity(doc.embedding, s.embedding) for s in
selected_docs], default=0)
        score = ÿ * relevance - (1 - ÿ) * diversity
        If score > best_score:
            best_score = score
            best_doc = doc
    Add best_doc to selected_docs
    Remove best_doc from candidate_docs
Return selected_docs
```

11.3.1.4 Hybrid Search (BM25 + Embedding based)

It combines traditional keyword-based BM25 search with vector embedding-based search.

BM25: Abbreviation for "Best Matching 25", it is an evolution of TF-IDF and a traditional information retrieval algorithm that evaluates the degree of keyword matching between a query and a document.

TF (term frequency): How often a word appears in a document

IDF (inverse document frequency): How rare a word is in the entire document

BM25 calculates search rankings by taking document length normalization into account in addition to TF and IDF.

Example problem: For the question "cancer treatment technology", BM25 can only rank documents that contain the keyword "cancer" repeatedly, whereas using only embeddings may select documents that are highly relevant but do not actually contain the keyword.

Solution: First filter with BM25, then reorder with vector embedding to simultaneously secure keyword-based consistency and semantic relevance.

Pseudocode:

Input: query, document_texts, document_embeddings, N, K

Step 1: top_n_docs = BM25(query, document_texts, top=N)

Step 2: embed_query = embed(query)

scores = [cosine_similarity(embed_query, embed(doc)) for doc in top_n_docs]

ranked_docs = sort_by_score_descending(top_n_docs, scores)

Return top K ranked_docs

Technique 11.3.1.5 Rerank

Rerank is a post-processing step performed to improve the accuracy of initial search results.

BM25 simply evaluates documents based on word frequency and consistency.

Example) Failure to understand context, meaning, word order, synonyms, etc. ÿ Incorrect results may appear at the top.

Models like BAAI's bge-reranker evaluate the semantic relevance between queries and documents.

Example) Considering contextual consistency, understanding intent, and diversity of expression ÿ Positioning more appropriate documents higher up.

The following shows an example of reranking after a first search.

pip install rank_bm25 transformers torch langchain

Full code:

```
from langchain.vectorstores import BM25Retriever
from langchain.schema import Document
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
import torch.nn.functional as F

# Document definition
documents = [
    Document(page_content="LangChain is a framework for developing LLM applications."),
    Document(page_content="BM25 is a ranking function used by search engines."),
    Document(page_content="Transformers library provides pretrained models."),
]

# Initial BM25-based retriever setup
retriever = BM25Retriever.from_documents(documents)
retriever.k = 5 # Top 5 searches

# Reranker model loading
tokenizer = AutoTokenizer.from_pretrained("BAAI/bge-reranker-v2-m3")
model = AutoModelForSequenceClassification.from_pretrained("BAAI/bge-reranker-v2-m3")
model.eval() # Inference mode
```

```
def rerank(query: str, docs: list[Document]) -> list[Document]:
    pairs = [(query, doc.page_content) for doc in docs]
    inputs = tokenizer(pairs, padding=True, truncation=True, return_tensors="pt")

    with torch.no_grad():
        scores = model(**inputs).logits.squeeze(-1) # (batch_size,)
        probs = F.softmax(scores, dim=0) # Soft scores can be used by probabilizing them.

    # Combine scores and documents
    reranked = sorted(zip(probs.tolist(), docs), key=lambda x: x[0], reverse=True)
    return [doc for _, doc in reranked]

# Search and Rerank
query = "How do LLM frameworks work?"
initial_results = retriever.get_relevant_documents(query)
final_results = rerank(query, initial_results)

# Output results
for i, doc in enumerate(final_results, 1):
    print(f"[{i}] {doc.page_content}")
```

11.3.1.6 HyDE (Hypothetical Document Embeddings)

This is a technique that first converts a question into a hypothetical answer and then searches documents based on the embedding of that answer.

Searching vectors using only "questions" can be insufficient. Using LLM, you can ask, "Write an answer to this question," then convert that hypothetical answer into a vector for use in searches. This allows you to enrich the context of your answers.

Multi-Query 11.3.1.7

This method generates queries with various semantic expressions from a single question, then performs individual searches for each query and integrates the results.

Input question: "What causes global warming?"

Diversifying queries using LLM:

"What are the main causes of climate change?"

"Why is the Earth's average temperature rising?"

"What is the relationship between greenhouse gases and global warming?"

Vector search by embedding each query into a vector

Generate responses by integrating search results **(into the RAG context)**

11.3.1.8 Embedding Vector Indexing

Indexing is required for high-speed search. For example, FAISS (Facebook AI Similarity Search) provides fast approximate nearest neighbor search for large vectors. This is similar to dense retrieval (DPR).
Used to store and retrieve vectors generated by the retriever.

Embedding vectors are typically located in a high-dimensional space with dimensions greater than 768 (based on BERT), and are semantically Similar texts have close directions in this space. These vectors are stored in a vector database such as FAISS.
It is indexed and stored, and when a query is entered, the embedded vector is used to find the nearest neighbor in the same way. Searching for vectors. This is a very similar principle to traditional spatial indexing.

FAISS supports various indexing structures and uses the following methods to quickly find similar vectors during queries:

1. Flat Index

Store all vectors as is and compare them all (accurate but slow)

IndexFlatL2, IndexFlatIP, etc.

2. IVF (Inverted File Index)

Divide vectors into K clusters (centroids) and search only vectors belonging to each cluster.

IndexIVFFlat, IndexIVFPQ, etc.

3. HNSW (Hierarchical Navigable Small World)

Graph-based search structure, allowing fast search by following only nearby neighbors.

IndexHNSWFlat

11.3.2 Augmentation

Combine the retrieved documents with the user's original question to create an LLM input prompt. Typically, the following prompt templates are used:

Example prompt template:

You are a helpful assistant. Use the following context to answer the question.

Context:

{retrieved_context}

Question:

{user_question}

Answer:

If the context is too long at this time, you need to chunk the document and select only a portion.

11.3.3 Generation

The prompt is passed to LLM (OpenAI GPT-4, Claude, LLaMA, etc.) to generate a response. The generation process is Follow these steps:

Forms an Augmented Prompt (user question + searched document).

Pass the corresponding prompt as input to LLM.

LLM repeatedly generates the next token and responds using pre-trained probabilistic language modeling. Create.

If necessary, adjust sampling parameters such as temperature, top-k, and top-p to increase generation diversity and quality. Adjust.

Generate pseudocode:

```
Input: prompt_text

Initialize output = ""

while not end_of_sequence:

    next_token = LLM.generate_next_token(prompt_text + output)

    output += next_token

Return output
```

11.3.4 Embedding and Chunking and Chunking)

11.3.4.1 outline

To make a document searchable in the RAG system, the document must first be broken down into processable units.

After dividing, each unit must be converted into a vector. This process is called "chunking".

This is called "embedding".

The reasons for chunking documents are as follows:

ÿ LLM or embedding models have a limited number of input tokens and cannot process long documents at once.

does not exist.

ÿ Splitting into meaningful paragraphs or sentences improves search quality without losing information.

ÿ By comparing in detailed units (chunks) at the search stage, the context most relevant to the question is identified.
You can find it.

ÿ When excessive information is input during augmentation generation, noise may occur, so cutting it into appropriate units and
using it contributes to improved performance.

11.3.4.2 Embedding

Each chunk is converted into a high-dimensional vector through an embedding model.

This vector quantifies the semantic characteristics of the text, and the semantic similarity can be calculated through the distance between the vectors.

The vectors are then stored in a search index such as FAISS, and compared to vectors generated by the same embedding model when a query is entered.

Pseudocode summarizing the embedding and chunking process:

```
#Chunking
chunks = split_document(doc, chunk_size=500, overlap=100)

# Embedding
for chunk in chunks:
    v = embed_model(chunk)
    vector_store.append(v)
```

Embedding quality directly affects retrieval accuracy, and chunking strategy provides appropriate context without information loss. It affects the maintenance.

11.3.4.3 Chunking

The entire document is usually too long to be embedded directly or passed to LLM. Therefore,

Split the document into chunks of a certain length (e.g. 300 to 1000 characters).

Apply overlap to avoid contextual disruption (e.g. 500 character chunks, 100 character overlap).

LangChain allows for flexible chunking by paragraph, sentence, or word using classes such as RecursiveCharacterTextSplitter.

In general, chunked text should be within the maximum token limit of the embedding model, which varies for each embedding model, but is typically as follows:

Embedding dimension (e.g. 512, 768, 1024)

- ÿ The number of dimensions of the fixed vector generated after text is embedded.
- ÿ Example: BERT-base has 768 dimensions

Maximum length of input text

- ÿ Usually 512 tokens (based on WordPiece and SentencePiece)
- ÿ If this limit is exceeded, the text will be truncated or an error will occur.

11.4 Internal Structure of the Augmented Prompt Chain

Frameworks like LangChain use the following chain structure, which allows for automated flow from document to question response.

Full chain components:

- **Document Loader ÿ Splitter:** Chunking PDF or text documents
- **Embedder:** Generates embeddings for each chunk.
- **Retriever:** Retrieving documents similar to question embeddings (Top-K or MMR)
- **Prompt Formatter:** Insert searched documents and questions into a prompt template.
- **LLM Generator:** Generate responses based on prompts

LangChain chain pseudocode:

Input: user_query

Step 1: embedded_query = embed(user_query)

Step 2: retrieved_chunks = retriever.retrieve(embedded_query)

Step 3: formatted_prompt = prompt_template.format(context=retrieved_chunks,
question=user_query)

Step 4: answer = llm.generate(formatted_prompt)

Return answer

In LangChain, this process can be bundled and used through the ConversationalRetrievalChain or RetrievalQA classes.

11.5 LangChain-based RAG example (based on PDF document)

Below is a full example of processing a PDF and answering questions using LangChain and the OpenAI API.

Reading and Chunking PDFs

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

loader = PyPDFLoader("my_doc.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
chunks = splitter.split_documents(docs)
```

Embedding and vector storage

```
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embedding_model)
```

Question-answer chain configuration

```
from langchain.chains import RetrievalQA
```

```
from langchain.chat_models import ChatOpenAI


retriever = vectorstore.as_retriever(search_type="mmr")
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    chain_type="stuff",
    retriever=retriever
)
```

Generate responses to questions

```
query = "What is the author's main argument in this document?"
result = qa_chain.run(query)
print(result)
```

11.6 Conclusion

RAG complements the limitations of LLM and enables the generation of reliable results by reflecting external knowledge and real-time information. In particular, it is a search-based QA system for unstructured documents such as PDF. This is useful when implementing. LangChain provides tools to modularize these chains and make them easy to configure.

12. Generating training data for RAG-based LLM models

12.1 How to create RAG training data

The data required for RAG-based LLM learning are Query, Context, and Answer.

It is a composed pair and can be created in the following way:

12.1.1 Leveraging public datasets

Public QA datasets such as Natural Questions, TriviaQA, and HotpotQA have a question-context-answer structure.

It is equipped with a RAG learning function.

It is frequently used in knowledge-intensive QA tasks based on Wikipedia.

12.1.2 Web crawling and document parsing

If domain-specific information is required, crawl relevant websites or documents and convert them into data.

Refining HTML and PDF documents to break them into paragraphs that can be vector-indexed, and manually querying the questions.

Or can be created through LLM

12.1.3 Synthetic Query Answer Generation (Synthetic QA Pairing)

Generate various queries through LLM based on existing documents and use parts of the documents as context.

Use it to construct the correct answer

Question-document pairs generated using HyDE (Hypothetical Document Embeddings) method are used for RAG learning.

use

12.1.4 Annotation Base writing Manual

For precise learning, experts or annotators write question-context-answer pairs directly.

High usability when building domain application systems

12.1.5 Document-Answer-Based Reverse Query Generation

When only documents and answers exist, a method of generating queries in reverse (LLM-based reverse query generation)

QA generation model or prompt tuning can be utilized.

12.2 Retrieval-Augmented Fine-Tuning (RAFT)

RAFT is a training method that complements the limitations of the existing Retrieval-Augmented Generation (RAG). Rather than simply including retrieved documents in the LLM input, it performs **supervised fine-tuning using examples containing the correct answer .** In other words, the model actually learns the correct answer based on the search results.

Key Features:

Feed the question and search document together to the model and learn the correct output.

Learning alignment between search results and correct answers

Fallback learning effect considering the case where the retriever is imperfect

Example flow:

For example in dataset:

```
question = example["query"]
gold_answer = example["answer"]
context_docs = retriever.search(question) # Use RAG data for training

input_text = format_prompt(question, context_docs)
output_text = gold_answer

loss = model(input_text, labels=output_text) # LLM model training
loss.backward()
optimizer.step()
```

RAFT significantly improves accuracy in QA tasks where the correct answer is clearly given or in knowledge-based response generation.

It can be improved and is effective in reducing the generation inconsistency and information distortion, which are limitations of RAG.

12.3 Conclusion

sLLM is a lightweight language model suitable for various environments, and to fine-tune it efficiently,

PEFT technique is mainly used rather than FFT. For large-scale learning, DDP, FSDP, and DeepSpeed are used.

Resources should be used efficiently by utilizing a distributed learning framework. In addition, RAG-based

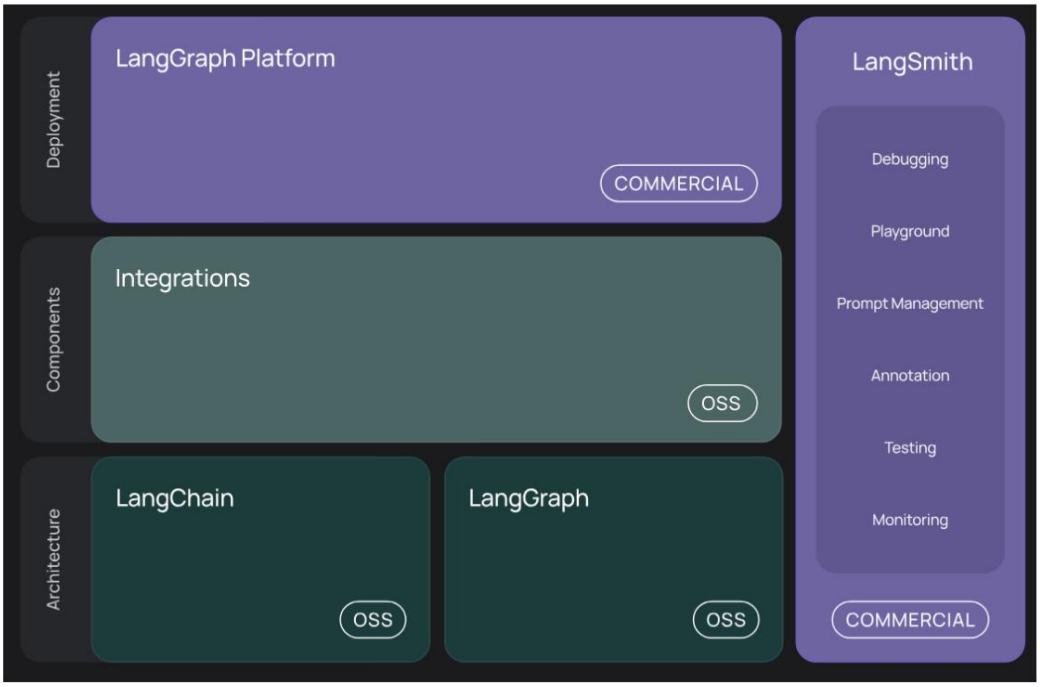
In the application, instead of simple retrieval-based responses, fine-tuning is performed through RAFT, and data in the question-context-answer structure is generated in various ways to simultaneously improve model accuracy and generalization.

It can be improved.

13. LangChain

13.1 Langchain Concept

LangChain is a framework designed to perform complex tasks by integrating large-scale language models (LLMs) with various tools and data sources. LangChain leverages LLMs' powerful natural language processing capabilities and solves problems through customizable chains and agents.



Introduction to LangChain

13.2 Installation and Example Code

Install the following in the terminal:

pip install langchain openai faiss-cpu tiktoken pypdf

Example code:

```
from langchain.chat_models import ChatOpenAI

# Create an OpenAI LLM
instance llm = ChatOpenAI(model="gpt-4", temperature=0.7, openai_api_key="your-api-key")
```

```
query = "What is LangChain?"
response = llm.run(query)
print(response)
```

13.3 Structure

LangChain's structure is divided into the following main components:

ÿ **LLM:** The language model itself.

ÿ **Prompt Templates:** Templates for formatting input data.

ÿ **Chains:** A workflow that sequentially connects multiple components.

ÿ **Agents:** Roles that dynamically select and execute tools.

13.4 Prompt Templates

Prompt templates are a crucial component of LangChain, structuring input data to maximize the performance of LLM.

Example code:

```
from langchain.prompts import PromptTemplate

# Create a prompt template
prompt_template = PromptTemplate( template="Provide
    a summary for the following topic: {topic}", input_variables=["topic"]

)

query = prompt_template.format(topic="LangChain's architecture") print(query)
```

13.5 LCEL (LangChain Expression Language)

LCEL is a simple, expressive way to connect LangChain components. Pipelines can be connected using the '|' operator to intuitively define workflows.

LCEL Example: Simple Chaining Using the '|' Operator

```
from langchain.prompts import PromptTemplate from
langchain.chat_models import ChatOpenAI from
langchain.output_parsers import StructuredOutputParser, ResponseSchema

# OpenAI LLM
llm = ChatOpenAI(model="gpt-4", temperature=0.7, openai_api_key="your-api-key")

# Prompt Template
prompt_template =

    PromptTemplate( template="{question}", input_variables=["question"]
)

# Output Parser
response_schema = ResponseSchema(name="answer", description="The final answer to the
question.")
parser = StructuredOutputParser.from_response_schemas([response_schema])

# LCEL Pipeline
query = "What are the key components of LangChain?" result
= prompt_template | llm | parser

# Execute
output = result.invoke({"question": query})
print(output)
```

13.6 Agent Mode

LangChain supports various agent methods, with the following main agent types:

13.6.1 Zero-shot ReAct

LLM works directly with a single prompt.

Example code:

```
from langchain.agents import initialize_agent from
langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(model="gpt-4", openai_api_key="your-api-key") agent =
initialize_agent(llm, tools=[]) # Create a basic agent query = "What is the
weather today?" response = agent.run(query)
print(response)
```

13.6.2 Conversational ReAct

As a conversational agent, it adapts based on previous conversations.

Example code:

```
from langchain.agents import ConversationalAgent from
langchain.memory import ConversationBufferMemory from
langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(model="gpt-4", openai_api_key="your-api-key")
conversation_memory = ConversationBufferMemory()
conversation_memory.save_context({"input": "What is LangChain?"}, {"output": "LangChain is a framework that
simplifies LLM tasks."}) agent =
ConversationalAgent(llm=llm, memory=conversation_memory)

query = "Can you explain more about LangChain's components?" response =
agent.run(query) print(response)
```

13.6.3 ReAct Docstore

Retrieves information from a document repository and then performs actions.

Example code:

```
from langchain.agents import DocstoreAgent
```

```
from langchain.document_loaders import LocalDocumentLoader from
langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(model="gpt-4", openai_api_key="your-api-key") document_loader
= LocalDocumentLoader(directory="./docs") agent = DocstoreAgent(llm=llm,
document_loader=document_loader)

query = "Summarize the content of the document." response =
agent.run(query) print(response)
```

13.6.4 Self-ask with Search

Breaking down questions into pieces generates self-research and responses. Self-Ask with Search, a technique proposed in a Google DeepMind paper (Lewis et al., 2022), breaks down complex questions into simpler ones, then retrieves and combines information from external sources (e.g., search engines) to provide answers.

LangChain implements this pattern as a combination of agent tool + LLM + search retriever.

Example code:

```
from langchain.agents import SelfAskWithSearchAgent from
langchain.tools import TavilySearchAPI from
langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(model="gpt-4", openai_api_key="your-api-key") search_tool =
TavilySearchAPI(api_key="your-tavily-api-key") agent =
SelfAskWithSearchAgent(llm=llm, search_tool=search_tool)

query = "What are the latest advancements in AI?" response
= agent.run(query) print(response)
```

13.7 How Agents Work

13.7.1 outline

chat-conversational-react-description is a representative agent provided by LangChain.
It is a type. This agent remembers the conversation history, makes inferences on its own, and selects tools.
It has a structure that directly generates final answers. It operates based on LangChain's standardized ReAct (Reasoning + Acting) pattern and is optimized to support natural conversational flow.

13.7.2 Action flow

The agent operates in the following order:

1. Receive the user's question (input).
2. Construct a prompt by integrating the conversation history (chat_history) and the current question.
3. Call LLM to determine the next action. The response will be one of the following:
   A. Use a specific tool (Action: tool name, Action Input: input)
   B. I will give you the final answer myself (Final Answer: answer text)
4. Parse the response into JSON format.
5. If the Action is tool usage, execute the tool and pass the result back to LLM for the next action. Decide.
6. If the Action is Final Answer, return the content to the user and end the conversation.
7. Record this question and answer in the conversation history and reflect it in the next question.

Agents can use the tool as many times as necessary until a Final Answer is reached, and then Reasoning is continuously performed throughout the process.

13.7.3 LangChain Configure internal prompts

When LangChain creates a chat-conversational-react-description agent, it internally uses a special Automatically generates a prompt template. This prompt consists of the following elements:

ÿ System message: "You are a helpful AI assistant."

ÿ Chat history (chat_history): Conversation history with past users.

ÿ Current question (input): The question newly entered by the user.

ÿ Tool name list (tool_names): List of available tools.

It also enforces a response format. The format is as follows:

Thought: (Contemplating what to do in this situation)
Action: (Name of tool to use)
Action Input: (value to be entered into the tool)

or

Final Answer:

By explicitly instructing LLM to follow this format, we maintain consistent behavior of the agent.

13.7.4 Prompt Creation Location

Based on the LangChain code, the prompt is created in the ConversationalChatAgent.create_prompt() function in the langchain/agents/conversational_chat/base.py file.

ÿ https://github.com/langchain-ai/langchain/blob/master/libs/langchain/langchain/agents/conversational_chat/prompt.py

This function synthesizes system messages, conversation history, the current question, and the list of tools to construct a final prompt and pass it to LLM.

13.7.5 Conclusion

Among LangChain agent designs, chat-conversational-react-description is the most versatile and conversationally natural structure. Based on the ReAct pattern, it iterates through actions (tool usage) and thoughts (thoughts), ultimately generating a Final Answer. LangChain automatically constructs internally structured prompt templates for this agent, ensuring consistent reasoning and action selection.

13.8 Basic Tools

LangChain provides a variety of built-in tools, including search, mathematical calculations, and Python code execution. For example, the search tool can be used as follows:

Example code:

```
from langchain.tools import TavilySearchAPI

# Tavily Search Tool
search = TavilySearchAPI(api_key="your-tavily-api-key")

query = "Latest advancements in AI" results =
search.run(query) print(results)
```

13.9 Vector DB and Search

LangChain integrates with vector databases for indexing and searching large-scale text data. This allows users to efficiently search embedded vectors.

Example code:

```
from langchain.vectorstores import FAISS

# Create and search a vector store
texts = ["LangChain is powerful.", "VectorDB is efficient."] vectorstore =
FAISS.from_texts(texts) query = "What is
LangChain?"
results = vectorstore.similarity_search(query) print(results)
```

Maximal Marginal Relevance (MMR) 13.9.1

The MMR algorithm is designed to increase the diversity of search results while maintaining relevance.

It's an algorithm.

• **Key Features:** o

Minimizes duplication with previously selected results to ensure diversity in the result set. o Maximizes relevance to the user's query. • **How it works:** o Selects the most relevant document

from the retrieved

document set. o Then, iteratively selects new documents by considering relevance

score and diversity. o The ÿ\lambda parameter controls the balance between relevance and diversity. •

Use Cases:

docsearch.as_retriever(search_type="mmr", search_kwargs={'k': 6, 'lambda_mult': 0.25}

) This setting returns 6 documents that are highly relevant to the query, but emphasizes diversity through the ÿ value.

13.9.2 Similarity Score Threshold

This algorithm retrieves documents only if the similarity score between the document and the query exceeds a certain threshold.

• **Key Features:** o

Control the quality of results through user-defined thresholds. o Filter out

documents with low similarity to improve search efficiency. • **How it works:** o

Calculates the

similarity score between the query and each document in vector space. o Selects only

documents that are above a set threshold. • **Use Cases:**

docsearch.as_retriever(

search_type="similarity_score_threshold",

```
    search_kwargs={'score_threshold': 0.8}
)
```

This setting returns only documents with a similarity score of 0.8 or higher.

13.9.3 Single Document Retrieval

It is an algorithm that searches for the most similar single document.

- **Key Features:**
  - o Quick and simple search is possible.
  - o Suitable for simple applications that require only a single result.
- **How it works:**
  - o Compute the document most similar to the query across the entire dataset.
  - o Returns only one document with the highest score.
- **Use Cases:**

```
docsearch.as_retriever(search_kwargs={'k': 1})
```

13.9.4 Filter-based Retrieval

Searches only documents that meet user-specified filter conditions.

- **Key Features:**
  - o Return only results that satisfy certain conditions (e.g. document properties).
  - o You can search for specific groups in large datasets.
- **How it works:**
  - o Apply custom filters when requesting a search.
  - o Only documents that meet the filter conditions are included in the search target.
- **Use Cases:**

```
docsearch.as_retriever(
    search_kwargs={'filter': {'paper_title': 'GPT-4 Technical Report'}}
)
```

13.9.5 RAG Retrieval QA Create a base chain

Retrieval-Augmented Generation (RAG) is frequently used in search-based QA systems. Below is

This is an example of creating a RAG QA chain.

• **procedure:**

1. **Retrieval:** Search for related documents in the vector DB.
2. **QA Chain:** The searched document is passed to the LLM (Language Model) to respond to the query.

Create. •

Code implementation:

```
def create_retrieval_qa(vectordb):
    retriever = vectordb.as_retriever(
        search_type="mmr",
        search_kwargs={'k': 6, 'lambda_mult': 0.25}
    )
    qa_chain = RetrievalQA.from_chain_type(
        llm=llm_model,
        retriever=retriever,
        chain_type="stuff"
    )
•   return qa_chain
```

The above code creates a QA chain based on the retrieved documents using the MMR algorithm.

• **Components:**

o search_type="mmr": Uses the MMR algorithm. o k=6: Returns the top 6 documents among the searched documents.

o lambda_mult=0.25: Balances relevance and diversity.

Conclusion 13.9.6

Various search algorithms in vector databases can be used in various applications according to their characteristics and operation methods. MMR is useful when both diversity and relevance are important, similarity score threshold returns quality-assured results, and filter-based search returns documents that meet specific conditions. You can search effectively.

13.10 Considerations

Exceptions to OpenAI LLM's token restrictions are frequent when building a RAG system utilizing LangChain.

This is a problem that occurs. In particular, if the document retrieved from the vector database is too long or the conversation

When the history accumulates and exceeds the maximum number of input tokens of the OpenAI API

An error like openai.BadRequestError occurs.

13.10.1 Token overflow when batch forwarding all documents to LLM

The chain_type="stuff" method passes all the searched documents to LLM as is, so the number of tokens is easily calculated.

It is exceeded.

Solution:

Change chain_type to map_reduce or refine to split and process documents.

Reduce document chunk size (chunk_size) and minimize overlap (chunk_overlap).

13.10.2 Token increase due to accumulated conversation history

ConversationBufferMemory stores all conversation history, so the longer the conversation, the more tokens it needs.

It increases.

Solution:

Summarize previous conversations using ConversationSummaryMemory.

Set max_token_limit to limit the maximum number of tokens to remember.

13.10.3 The number of documents retrieved exceeds the total prompt length.

If there are many chunks retrieved from FAISS, etc., the total document length transmitted to LLM will increase, causing errors.

It happens.

Solution:

Reduce duplication by reducing n in search_kwargs={"k": n} or adjusting lambda_mult, etc.

Summarize and deliver the retrieved documents using ContextualCompressionRetriever, etc.

13.10.4 Prompt configuration without considering the maximum number of tokens in the model

GPT-3.5-turbo has an input limit of 4,097 tokens, and GPT-4 has an input limit of 8,192 to 32,768 tokens.

Solution:

Explicitly specify max_tokens when creating LLM.

Pre-calculate and limit the length of input documents and question tokens.

13.10.5 Pre-adjustment is difficult due to difficulty in calculatingthe number of tokens.

There is feedback that it is difficult to accurately calculate the number of LLM input and response tokens.

Solution:

Calculate the number of prompt tokens using the tiktoken library.

If necessary, filter chunks after calculating tokens and if they exceed a certain standard.

13.10.6 Other

When implementing the RAG system through LangChain, the design takes into account the token limitation of the OpenAI API.
It is essential. The above issues are the main causes of exceptions due to token length exceeding, and proper chain configuration
This can be prevented through preprocessing. In particular, functions such as map_reduce, memory summarization, token
length measurement, and retriever filtering are key solutions to the token overflow problem.

When considering future model scalability or high-cost API usage, it is important to detect these issues in advance.
Structural management helps increase the reliability and maintainability of the RAG architecture.

13.11 Agent Tools Other than Langchain

The tools listed in the table below each help develop and manage AI agents in different ways.
It can be given, and the use cases may vary depending on the characteristics of each tool.

Tool name	explanation	link
AutoGPT: An	AI agent that automatically performs multiple tasks using text-based input. GPT	AutoGPT

	Designed to automate a variety of tasks using models.	GitHub
Pydantic	A data validation and configuration tool for building AI agents and models. Pydantic allows you to structure and validate the inputs and outputs of AI models.	PydanticAI
LlamaIndex	Llama model is used to implement efficient data search and agent functions. Tools, particularly strong in processing and searching large-scale text data.	LlamaIndex GitHub
CrewAI is a tool	for managing AI agents centered on collaboration. It connects multiple AI agents. Useful for solving complex problems.	CrewAI Website
LangGraph is an	AI agent management tool based on graph database, which manages complex data relationships. Helps you process and navigate effectively.	LangGraph GitHub

13.12 Conclusion

LangChain is a powerful framework leveraging LLM, offering a variety of features to simplify and efficiently process complex workflows. LCEL and agent methods can be customized to suit the user's needs.

It is particularly useful in that it can be flexibly configured and executed.

13.13 Reference

1. **LangChain Docs:** Code for all modules, from chains to memory and agents
   Very clean documentation with examples.
2. **LangChain Python GitHub:** Check out what's happening under the code or get the latest
   If you want to track updates.
3. **Hugging Face Transformers:** Deep understanding of transitioning to embedding models,
   local LLM, or Transformers pipelines.
4. **Facebook AI's FAISS Guide:-** If you want to scale your vector store or deploy it at scale.
5. **LangChain + RAG Full Stack Tutorial:** Practice code provided directly by the team.
6. **RAG in Production — Pinecone Blog:** In a production context, including challenges and best practices
   Describe the RAG architecture.
7. **Harrison Chase (LangChain):** Continuous updates, tips, and sneak peeks at future features.
8. **Retrieval-Augmented Generation** Description:- How RAG fits into the broader LLM ecosystem
   An introduction to suitability.

14. ollama

14.1 Overview

Ollama is a lightweight runtime that helps you run and utilize large-scale language models (LLMs) locally. It's designed to load open source models like GPT, LLaMA, and Mistral directly into your local environment for use in things like prompt response and RAG search. It works without Docker and integrates easily with LangChain.

14.2 How to Install Ollama on Windows

https://ollama.com Connect to .

Select your operating system (Windows) and download the installer (.exe).

After installation, open Command Prompt (CMD) or PowerShell and check if the ollama command is recognized.

14.3 Key Terminal Commands

```
# Download and run the model
ollama run llama3


# Check the list of installed models
ollama list


# Terminate the running model
ollama stop


# Delete a model locally
ollama remove llama3
```

The model is automatically downloaded the first time it is run, and is cached locally thereafter, eliminating the need for repeated downloads.

14.4 Using Ollama in LangChain

To call Ollama from LangChain, the following environment must be configured:

Install required packages

```
pip install langchain langchain-community
```

Python example code

```
from langchain.llms import Ollama

llm = Ollama(model="llama3")
prompt = "What are the characteristics of the LLaMA3 model run on Ollama?"
response = llm.invoke(prompt)
print(response)
```

This code sends a prompt to the locally running llama3 model and outputs a text response. Ollama operates as a REST API via port localhost:11434 by default, and LangChain handles the prompt request through this API.

Ollama can run on a CPU without a GPU. However, performance is significantly lower than on a GPU, and may be affected by responsiveness and memory usage. Lightweight models like llama2 and mistral can run without a GPU, but models larger than 7B may experience slower loading and inference times.

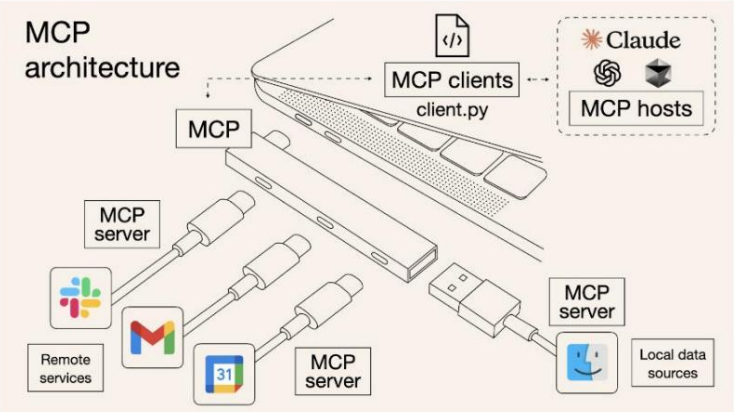
15. MCP (Model Context Protocol) and AI Agent Development

15.1 Overview and Concepts

The Model Context Protocol (MCP) defines how various applications provide context to the LLM.

An open protocol proposed by Anthropic, the developer of Claude, to standardize MCP. MCP connects various tools and data to AI models in an integrated manner, like a USB-C port, and allows AI agents to use tools.

It provides a standard interface required for recognition and use.



The introduction of MCP in LLM-based applications goes beyond a simple question-and-answer model and involves external systems.

It provides a foundation for expanding into a true "agent" role that can interact with others.

Flexible integration, data protection, and inter-vendor portability are guaranteed. For example, LLM can be accessed through MCP.

You can open and read local files, retrieve information through external APIs, and handle complex user requests.

there is.

15.2 MCP, ACP, A2A

15.2.1 introduction

The development of artificial intelligence agent systems no longer relies on the capabilities of a single model.

Tools and data sources, multiple agents located on different platforms, collaborate organically.

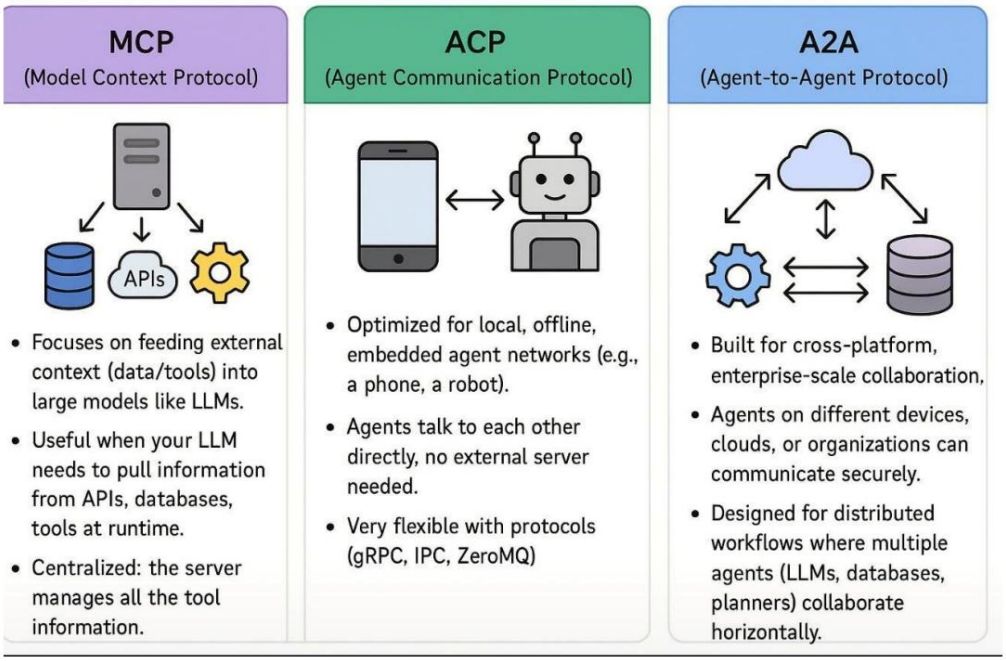
Structure is emerging as a new paradigm. Ensuring efficient communication in such an environment

Various agent protocols have emerged for this purpose, and the representative ones are Model Context Protocol (MCP), Agent

Communication Protocol (ACP), and Agent-to-Agent Protocol (A2A). Although these protocols overlap functionally,

they differ in design purpose, scope of application, and technical

There are differences in the implementation method.



MCP, ACP, A2A concept diagram

15.2.2 MCP (Model Context Protocol)

Model Context Protocol (MCP) was proposed in mid-2023 by Anthropic, known as the developer of Claude.

MCP is a communication framework optimized for connecting external data sources or tools in an architecture centered around a Large Language Model (LLM), primarily for tool invocation and context.

It is designed for injection. Essentially, MCP is a tool that provides LLM with appropriate information for a user-entered query.

It provides a mechanism to retrieve information in real time from external databases, APIs, file systems, etc., so that responses can be generated.

Technically, MCP follows a centralized architecture. All tool calls and resource accesses are processed through a specific MCP server, which exchanges tool lists, descriptions, parameter schemas, and other information with LLM via JSON-RPC. While this approach ensures both scalability and reliability, it raises concerns about overall system performance degradation in the event of server failure or bottlenecks. The Claude desktop client is a representative MCP hosting environment, and various Python-based tools are registered and operated via the mcp SDK.

15.2.3 ACP (Agent Communication Protocol)

Agent Communication Protocol (ACP) is known to have been proposed by Anthropic and some community-based projects as a complementary protocol after MCP, especially in local agent networks.

It aims for efficient communication. Unlike MCP, ACP is structurally different from MCP in that it does not require a central server.

It has a difference. It enables standalone agent-to-agent interaction without an internet connection, and it works in offline environments such as smartphones, robots, and embedded devices.

Technically, ACP is a high-performance lightweight framework like gRPC, IPC (Inter-Process Communication), and ZeroMQ.

It is compatible with communication protocols, and each agent can send and receive messages directly. Therefore, it can establish a communication path with low message delay and relatively secure security.

The structure is highly useful for collaboration between IoT devices or robots, and asynchronous AI processing in mobile environments.

15.2.4 A2A (Agent-to-Agent Protocol)

Agent-to-Agent Protocol (A2A) is expected to be implemented by multiple organizations, including Microsoft, Hugging Face, and OpenAI, starting in 2024.

A2A is an open standard that has begun to be discussed in the corporate and open source communities.

It aims to create a horizontal communication framework that allows multiple agents to cooperate with each other in an environment. In particular,

This protocol is designed with cross-platform collaboration between enterprises or cloud-to-edge in mind.

It is designed with security, authentication, and distributed work architecture as its core features.

A2A is ideal for collaborative workflows between LLMs, knowledge databases, and planners distributed across different networks or organizations. It supports OAuth2-based authentication, TLS encryption, and multi-node support.

Includes namespace negotiation, enterprise-grade system integration, digital twins, and real-time collaboration.

It can be used in AI systems.

15.2.5 comparison

MCP, ACP, and A2A are agent communication protocols designed for their respective purposes and technical requirements, and can be used in parallel in a hierarchical or cooperative manner, rather than simply as replacements. MCP

Optimized for utilizing external tools centered on a single LLM, ACP is a network-independent local

It supports agent collaboration. On the other hand, A2A provides security between multiple agents in large-scale distributed environments.

Enables collaboration.

MCP is an “input interface” that connects tools and data to a single LLM.

ACP acts as a “direct communication line between agents” in the local environment.

A2A is a “horizontally distributed collaboration infrastructure” for complex inter-organizational work.

These protocols can serve as platforms for real-time collaboration in future multi-agent AI systems, automated tool orchestration, and integration across diverse computing environments.

15.3 MCP Architecture Structure and Communication Method (Stdio vs. SSE)

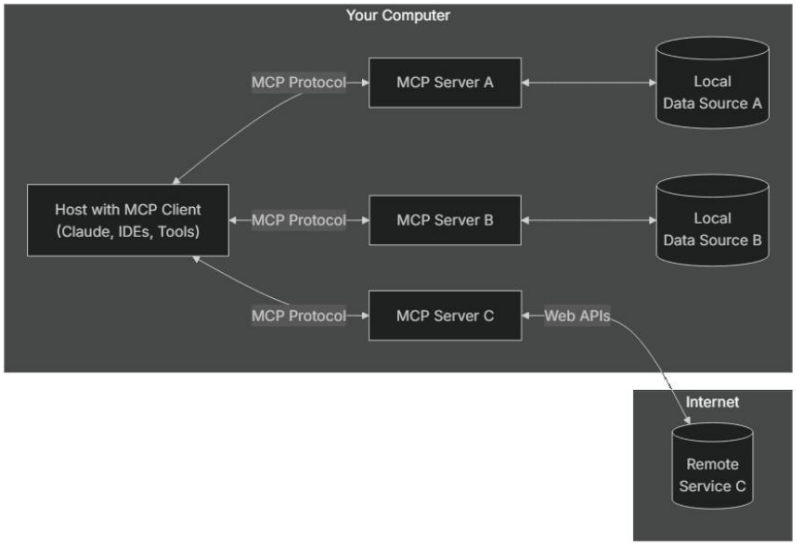
MCP follows a client-server architecture and consists of the following elements:

MCP Host: An integrated environment for MCP clients, including the Claude desktop, AI agents, and cursor IDE.

MCP Server: A program that exposes functionality and handles tool registration and execution. It operates in various runtime environments, including Python and Node.js.

Local data sources: local resources such as file systems and DBs

Remote services: API-based external services such as GitHub and Brave Search.



15.4 MCP Server Execution Mode

stdio (standard input/output):

The MCP server receives commands through stdin and responds to stdout.

Suitable for local single-process communication and is useful for debugging and simple execution.

SSE (Server-Sent Events):

HTTP-based asynchronous event streaming

/sse The server pushes data to the client via a persistent connection on the endpoint.

Suitable for multi-client and long-running applications

item	stdio mode	SSE mode
Communication method stdin/stdout		HTTP + EventStream
merit	Simple implementation, local fit, real-time streaming, scalability	
Use case testing, local tool execution, real-world service, LLM integration		

15.5 Protocol Message Structure (JSON-RPC)

MCP follows the JSON-RPC 2.0 standard. The following is an example of calling the MCP tool:

request

```
{
  "jsonrpc": "2.0",
  "id": "call-1",
  "method": "callTool",
  "params": {
    "name": "create_record",
    "arguments": {
      "title": "New record",
      "content": "Record content"
    }
  }
}
```

response

```
{
  "jsonrpc": "2.0",
  "id": "call-1",
```

```
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Created New Record"
      }
    ]
  }
}
```

This structure standardizes function calls, resource responses, error handling, etc.

15.6 Designing and Calling MCP Tools

Tools are defined based on JSON Schema within the MCP server.

ÿ Reference: Model Context Protocol (MCP) Anthropic Development Method - WikiDocs

example:

```
@app.list_tools()
async def list_tools():
    return [
        types.Tool(
            name="calculate_sum",
            description="Tool to add two numbers",
            inputSchema={
                "type": "object",
                "properties": {
                    "a": {"type": "number"},
                    "b": {"type": "number"}
```

```
                },
                "required": ["a", "b"]
            }
        )
    ]
```

Calling the tool:

```
@app.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "calculate_sum":
        return [types.TextContent(type="text", text=str(arguments["a"] + arguments["b"]))]
```

15.7 MCP Installation and Development Practice (Python-based)

Basic installation:

```
pip install mcp pydantic-ai fastmcp tavily-python
```

Calculator tool example:

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("SimpleCalc")

@mcp.tool()
def add(a: int, b: int) -> int:
    return a + b

if __name__ == "__main__":
    mcp.run()
```

Running the server:

```
mcp dev mcp_simple_calc.py
```

15.8 Client Code Example

Stdio method:

```
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


server_params = StdioServerParameters(
    command="python",
    args=["./mcp_simple_calc_server.py"]
)


async def run():
    async with stdio_client(server_params) as streams:
        async with ClientSession(*streams) as session:
            await session.initialize()
            tools = await session.get_tools()
            print("Tool list:", tools)
            result = await session.run("Add 2 and 4")
            print("Response result:", result.data)
```

SSE method:

```
from mcp.client.sse import sse_client
from mcp import ClientSession


async def run():
    async with sse_client(url="http://localhost:5173/sse") as streams:
        async with ClientSession(*streams) as session:
```

```
await session.initialize()

tools = await session.get_tools()

print("Tool list:", tools)

result = await session.run("calculate power 2, 4?")

print("Response result:", result.data)
```

15.9 Ollama + MCP

Clone the repository:

```
git clone https://github.com/chrishayuk/mcp-cli

cd mcp-cli
```

Installing dependencies:

```
pip install uv

uv sync --reinstall
```

Running Ollama:

```
ollama run llama3.2
```

Running the MCP CLI:

```
uv run mcp-cli chat --server filesystem --provider ollama --model llama3.2
```

Ollama enables MCP tool discovery and invocation in a local LLM environment, and allows tool-based AI agents to run without the internet.

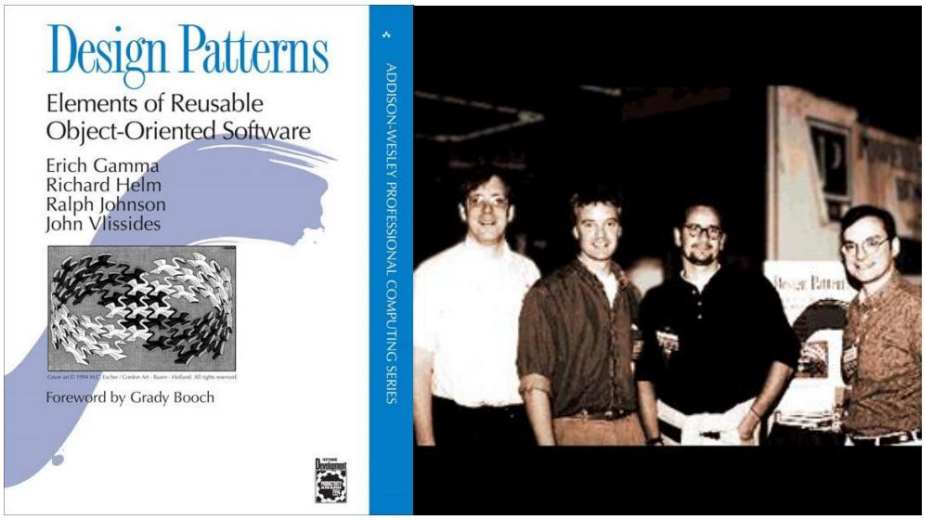
15.10 Conclusion

MCP is a protocol that plays a key role in functionally expanding LLM-based systems. Its standardized tool invocation, support for various runtimes, JSON-RPC-based communication, and integration of local and remote tools will form the foundation for future agent-based AI system design.

16. AI Agent Design Patterns

16.1 Design Patterns for AI Agents

The concept of design patterns was first established in 1994 by the "Gang of Four (GoF)"—Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides—in their book Design Patterns: Elements of Reusable Object-Oriented Software. Its starting point was the patternization of solutions to recurring problems in object-oriented software design. They separated the relationships and responsibilities of each element in OOA/D (Object-Oriented Analysis/Design), forming the basis for constructing an architecture that is easy to maintain and expand.



This philosophy also applies to AI agent design. In particular, recent LLM-based agents are evolving beyond single-model calls into complex systems that include document retrieval (RAG), external tool integration (MCP), multi-LLM collaboration, and multimodal processing. Therefore, a systematic approach to combining these is needed.

Establishing the "Agent Design Pattern" is a competency that DX consultants and AX (Agent Experience) designers must understand, and can serve as the basis for planning and realizing unique brand agents.

16.2 Classification and Principles of AI Agent Design Patterns

The following are key AI agent design patterns, organized based on real-world implementation examples and structural features.

This is an example. Branding can be done in this way depending on the purpose of the organization or individual.

16.2.1 Single-Skill Pattern (Single-Skill Agent)

This pattern is used when building agents specialized for a single task, for example, in-house document-based
It consists of a fine-tuned LLM or RAG-based architecture, focusing on a single function such as FAQ search, weather response, or email
summary. The input is a simple query, which is processed by an embedding retriever and a single LLM.
It is composed of the most basic form of agent design and is used to quickly service a single function.
It's suitable.

16.2.2 Multi-Skill Routing Pattern (Multi-Tool Routing Agent)

This pattern is a structure that selectively executes various tools or skills depending on the input. For example,
For example, the command "calculate" uses the Python execution tool, "tell me the weather" uses the external API tool, and "document" uses the
"Summarize" is a way to call LLM. It can be implemented in MCP-based tool systems, OpenAI Function/Tool Calling, LangChain ReAct, etc.,
and the client classifies the input as a selector (Classifier or
It has a built-in Planner.

16.2.3 Expert Ensemble Agent

This pattern combines LLMs or agents specialized in different domains to achieve a cooperative response.
For example, medical questions can be routed to Med-PaLM, legal questions to GPT-Legal, or all
It is a method of combining answers from experts based on majority vote or score. Multi-agent
It is an advanced strategy that combines and is suitable for professional-based services.

16.2.4 Multimodal Retrieval Agent Pattern

In addition to text, it simultaneously supports various formats of data such as images, voice, video, PDF, PPT, and music.
This structure is designed for searching and analyzing. For this purpose, multimodal embedding models (such as CLIP, BLIP, Whisper, and
MusicLM) are used, and documents are searched based on similarity in a unified vector search index. This is an essential
pattern for complex input-based services, such as visual and audio-based meeting summaries.

16.2.5 Meta Agent Pattern (Meta Agent)

It is a structure in which an agent can control or combine other agents. The user's complex
Given an instruction ("Summarize the document and create a PPT"), we first call the summary tool, and then, based on the results,
It consists of a series of workflows that re-invoke the generation tool. This pattern can be implemented in Self-RAG, Planner + Tool
Executor, ReAct-based architectures, etc.

Pattern 16.2.6 Context Enrichment (Context Enrichment Agent)

This pattern augments the LLM's input context with external real-time information (APIs, databases, cloud data, etc.). Primarily based on the Model Context Protocol (MCP) architecture, it obtains real-time information from various server-provided tools and inserts it into the LLM's input. For example, it might call an exchange rate API and then pass the results to the LLM along with a user query.

16.3 Applying Development Strategy and Branding Perspectives

AI agents aren't simply built with high-performance LLMs. They must be built based on design patterns tailored to user needs, industry characteristics, and information architecture. This ensures brand credibility and expertise.

AX (Agent Experience) designers must selectively combine the above patterns according to user scenarios and interaction flows to flesh out the UX design. Conversely, DX consultants must be able to interpret the structure between business architecture and technical elements and design optimized data flows and module configurations. Beyond simple API integration, they must consider how multiple LLMs and tools can harmoniously collaborate and leverage their respective strengths from a branding perspective.

For example, if you are planning a brand called 'Legal Expert Consulting Agent,' you need a structure that combines expert group patterns (law, contracts, labor law, etc.) + context enhancement patterns (real-time legal database) + multimodal patterns (understanding contract PDFs).

If the LLM model you are using is in a business domain that does not have token embeddings for knowledge, separate model learning development, such as fine-tuning, is required.

16.4 Conclusion

AI agent systems are essentially a combination of multiple components, and the design of how each component will cooperate and what responsibilities it will have is key. Object-oriented design patterns
Just as the 1990s transformed the landscape of software engineering, AI agent design patterns will become a central principle defining user experience and service architecture in the AI era.

These AI agent design patterns will become more and more concrete as the LLM platform ecosystem develops in the future. The ability of DX strategists and AX designers to flexibly combine and evolve these patterns can become a key competitive edge for brand differentiation.